BASI DI DATI

Architettura fisica dei DBMS

Daniele Riboni · Università degli Studi di Cagliari
Dipartimento di Matematica e Informatica

Slide tratte in parte dal materiale didattico del libro: Atzeni, Ceri, Fraternali, Paraboschi, Torlone. Basi di dati. McGraw-Hill

MATERIALE DIDATTICO

Materiale

Atzeni, Ceri, Fraternali, Paraboschi, Torlone. "Basi di dati". Quinta edizione. McGraw-Hill, 2018

Capitolo 11

oppure

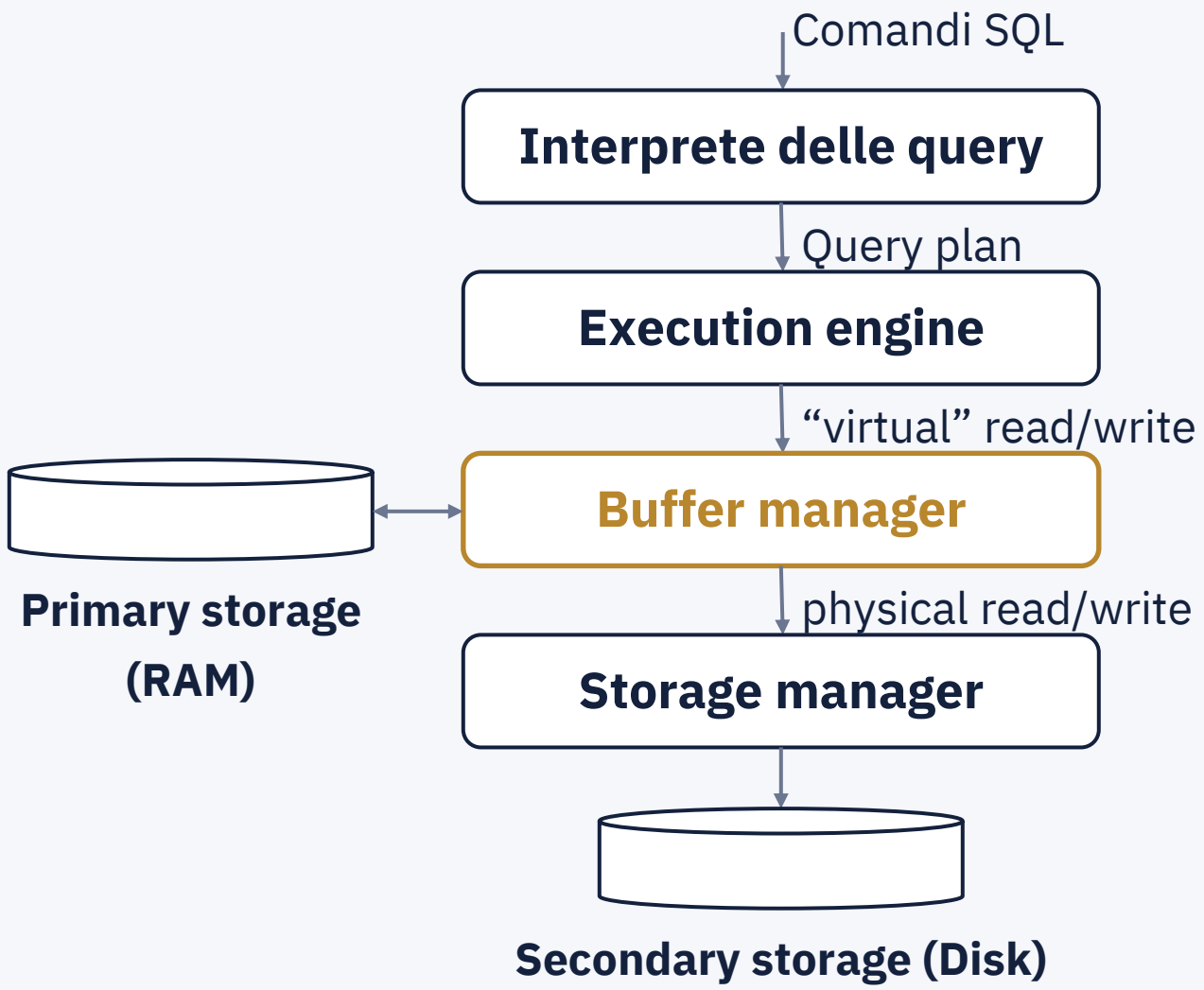
Garcia-Molina, Ullman, Widom. "Database Systems: The Complete Book". 2/E. Pearson, 2009

Sections 13.1 to 13.2, 13.5 to 13.8, 14.1 to 14.3, 15.1 to 15.4.1

LIVELLO FISICO DEL DBMS

Gestione della memoria

Architettura di alto livello del DBMS



MEMORIA SECONDARIA

Memoria principale e secondaria

I programmi possono fare riferimento solo a dati in **memoria principale**

Le basi di dati debbono essere scritte in memoria secondaria per due motivi:

- Dimensioni
- Persistenza

I dati in memoria secondaria possono essere utilizzati solo se prima trasferiti in memoria principale

MEMORIA SECONDARIA

Memoria principale e secondaria

I dispositivi di memoria secondaria sono organizzati in **blocchi** di lunghezza fissa (alcuni KB)

Le uniche operazioni sui dispositivi sono **la lettura e la scrittura di un intero blocco**

Pagina: unità di suddivisione logica della **RAM**

Per semplicità assumiamo: **dimensione pagina = dimensione blocco**

Tempi di accesso alla memoria secondaria

- **Hard disk tradizionale (HDD):**
  - tempo di posizionamento della testina, tempo di latenza, tempo di trasferimento...
  - **In media ~ 1 - 10 ms**
- **Memoria secondaria con unità a stato solido (SSD)**
  - Più costosa, ma tempi di accesso circa 50 volte minori rispetto a HDD
- **Accesso a memoria terziaria (tapes, optical jukebox)**
  - **In media ~ 5 - 60 secondi**

MEMORIE

Memoria principale e secondaria

Il costo di un accesso a memoria secondaria è quattro o più ordini di grandezza maggiore di quello per operazioni in memoria centrale

Semplifichiamo:

il costo di una operazione del DBMS dipende esclusivamente dal numero di accessi a memoria secondaria

(numero di blocchi che devono essere letti o scritti)

Accessi a blocchi “vicini” costano meno **(contiguità)**

IL BUFFER E IL SUO GESTORE

Buffer manager

Buffer

area di memoria centrale, gestita dal DBMS (preallocata) e condivisa fra le transazioni

"deposito temporaneo" delle porzioni di DB correntemente utilizzate

Scopo del buffer manager

Ridurre il numero di accessi alla memoria secondaria

BUFFER MANAGER

Dati gestiti dal buffer manager

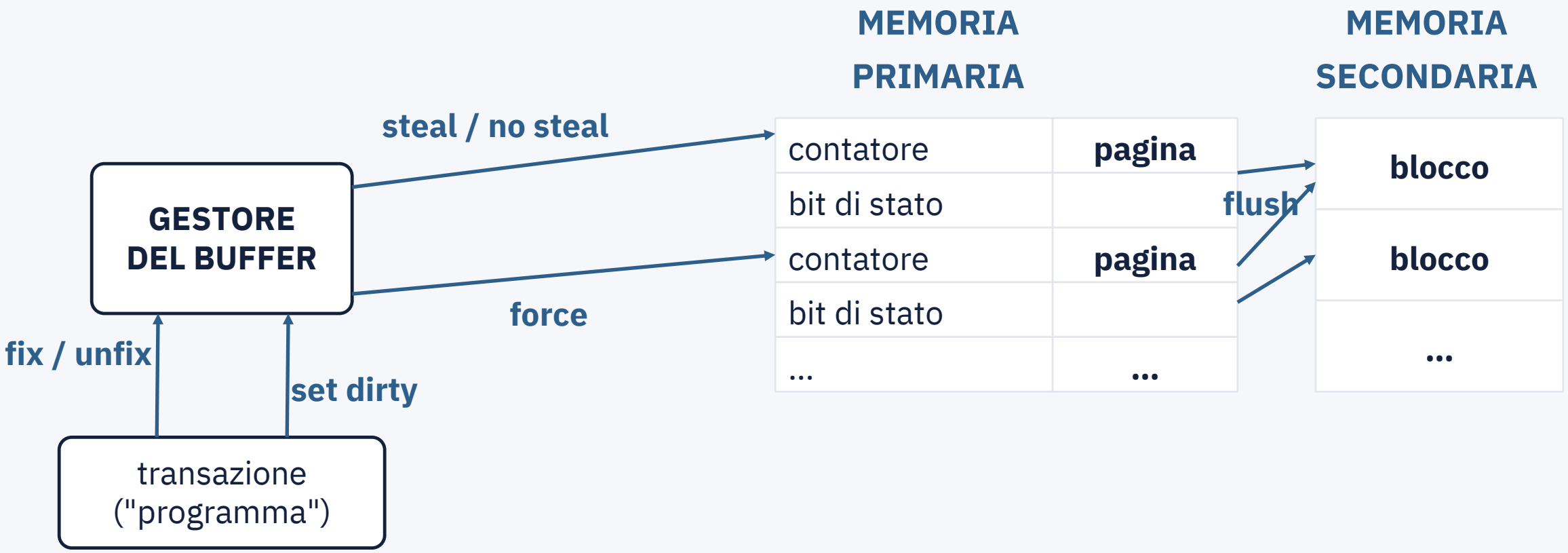
- Il contenuto del buffer
- Un direttorio che per ogni pagina mantiene
  - il file e il numero del blocco corrispondente in HD
  - due variabili di stato:
    - un contatore che indica quanti programmi utilizzano la pagina
    - un bit di stato che indica se la pagina è "sporca", cioè se è stata modificata (la scrittura su memoria secondaria può essere differita)

Funzioni del buffer manager

- Riceve richieste di lettura e scrittura di blocchi (che contengono record)
- Utilizza il buffer quando possibile
- Esegue le richieste accedendo alla memoria secondaria solo quando indispensabile

- Esegue le primitive: `fix, unfix, setDirty, force`

Gestione del buffer



Interfaccia offerta dal buffer manager

- `fix`: richiesta di una pagina
  - richiede una lettura in memoria secondaria solo se la pagina non è nel buffer
  - incrementa il contatore associato alla pagina
- `setDirty`: comunica al buffer manager che la pagina è stata modificata

Interfaccia offerta dal buffer manager

- `unfix`: indica che la transazione ha concluso l'utilizzo della pagina
  - decrementa il contatore associato alla pagina
- `force`: trasferisce in modo sincrono una pagina in memoria secondaria
- `flush`: scrittura asincrona in memoria

BUFFER MANAGER

Esecuzione della fix

- Cerca la pagina nel buffer
  - Se c'è, restituisce l'indirizzo
- Altrimenti, cerca una pagina libera nel buffer (avente contatore a zero) e la sovrascrive
  - Se ce n'è più di una? Politica FIFO, oppure Least Recently Used, o altra politica...
  - La pagina sovrascritta viene prima salvata su disco se necessario

Esecuzione della fix

E se non ci sono contatori a zero?

Due alternative

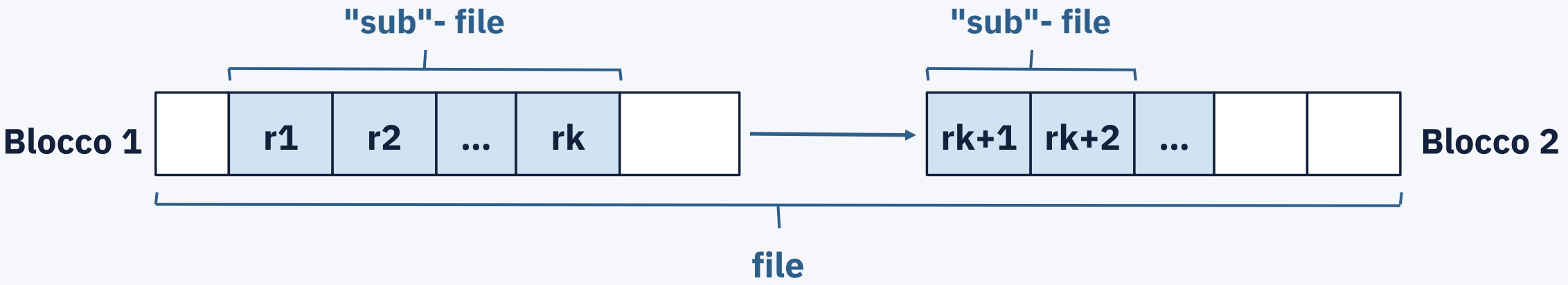
- **"steal"**: selezione di una "vittima" (pagina occupata del buffer)
- **"no-steal"**: l'operazione viene posta in attesa

FILE E RECORD

DBMS e file system

- L'organizzazione interna dei file è gestita direttamente dal DBMS
- struttura all'interno dei singoli blocchi
- distribuzione dei record nei blocchi

Blocchi e file



rx = record

FILE E RECORD

Serializzazione dei record

Alternative:

- Formato dei record fisso oppure auto-descrittivo
- Lunghezza dei campi fissa oppure variabile

SERIALIZZAZIONE

Formato fisso e lunghezza fissa

Employee record

(1) E#, 2 byte integer

(2) E.name, 10 char.

(3) Dept, 2 byte code

Schema

(nel direttorio)

Record:

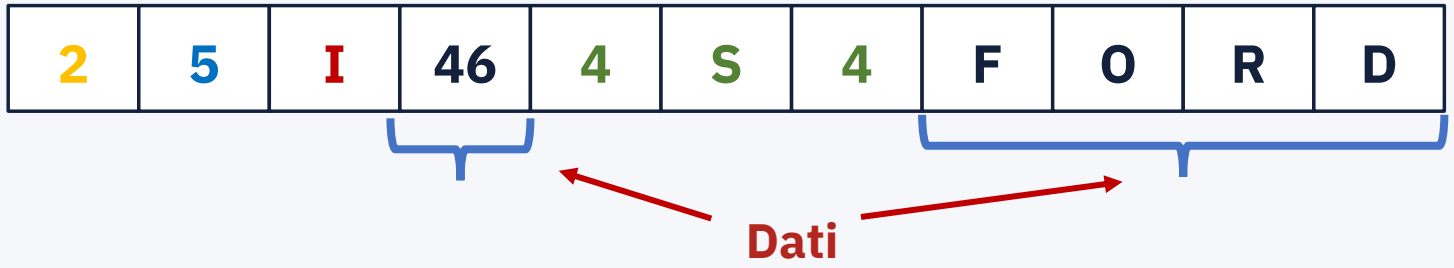
55	s m i t h	02
83	j o n e s	01

Record

(in altri file)

SERIALIZZAZIONE

Formato auto-descrittivo e lunghezza variabile



Vantaggi e Svantaggi?

`2` → numero di campi

Metadati

`5` → codice o nome della prima colonna

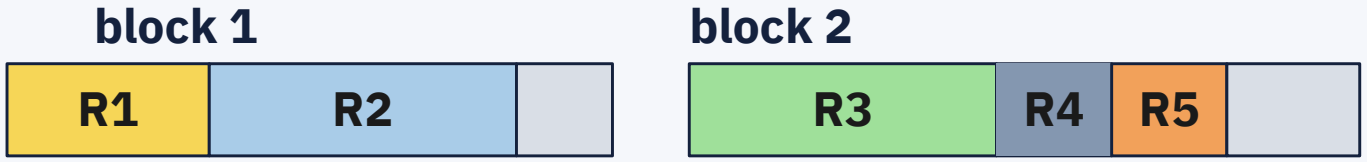
`I` → Integer type

`4, S, 4` → codice 2° colonna, String type, lunghezza stringa

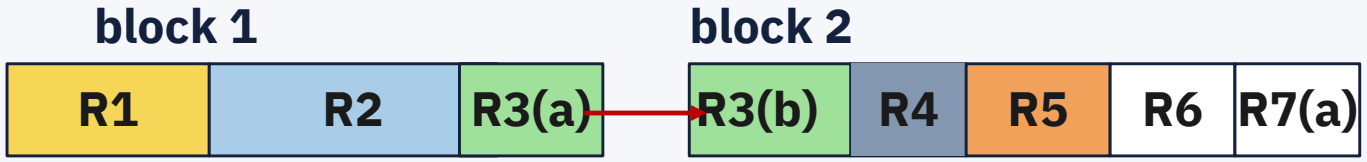
`F O R D` → valore della stringa (“FORD”)

Scrittura su disco: Unspanned vs Spanned

Unspanned: ogni record sta in un solo blocco



Spanned: record può essere diviso in più blocchi



FILE E RECORD

Spanned vs. unspanned

Unspanned molto più semplice da implementare e veloce, ma determina perdita di spazio

Spanned necessario per record più grandi di un blocco

ORGANIZZAZIONE FISICA

Row vs Column Store

I formati visti in precedenza scrivono su disco in ordine di record *(row store)*...

Altra opzione: scrivere in ordine di colonna ***(column store)***

ORGANIZZAZIONE FISICA

Row Store

Esempio: Relazione "Acquisto"

`Acquisto(id, cust, prod, store, price, date, qty)`

	id	cust	prod	store	price	date	qty
BLOCK1	1	Rossi	Mouse	MI	12	10/1	2
	2	Bianchi	Tastiera	TO	25	11/1	1
BLOCK2	3	Verdi	Monitor	RM	150	12/1	1
	4	Russo	Mouse	MI	12	13/1	3
BLOCK3	5	Rossi	Monitor	TO	150	14/1	2
	6	Bianchi	Tastiera	RM	25	15/1	1

Ogni blocco contiene tuple intere e consecutive (qui 2 per blocco).

ORGANIZZAZIONE FISICA

Column Store

Esempio: Relazione "Acquisto"

`Acquisto(id, cust, prod, store, price, date, qty)`

BLOCK1
(colonna id)

1
2
3
4
5
6

BLOCK2
(colonna cust)

Rossi
Bianchi
Verdi
Russo
Rossi
Bianchi

BLOCK3
(colonna prod)

Mouse
Tastiera
Monitor
Mouse
Monitor
Tastiera

Gli ID possono essere scritti esplicitamente
oppure essere impliciti in base all'ordine (come qui: la riga N di ogni blocco è la stessa tupla)

ORGANIZZAZIONE FISICA

Column Store - variante

Esempio: Relazione "Acquisto"

`Acquisto(id, cust, prod, store, price, date, qty)`

BLOCK1
(colonna id)

```
1
2
3
4
5
6
```

BLOCK2
(colonna cust)

```
(1, Rossi)
(2, Bianchi)
(3, Verdi)
(4, Russo)
(5, Rossi)
(6, Bianchi)
```

BLOCK3
(colonna prod)

```
(1, Mouse)
(2, Tastiera)
(3, Monitor)
(4, Mouse)
(5, Monitor)
(6, Tastiera)
```

Gli ID devono essere espliciti

(ogni valore è marcato con l'id della tupla a cui appartiene)

ORGANIZZAZIONE FISICA

Row vs Column Store

Vantaggi del Column Store

Letture efficienti per applicazioni di "data mining" e statistiche

Vantaggi del Row Store

Più efficienti per letture e scritture di molteplici campi dello stesso record

Nel seguito assumiamo di usare Row Store

ORGANIZZAZIONE FISICA

Strutture sequenziali

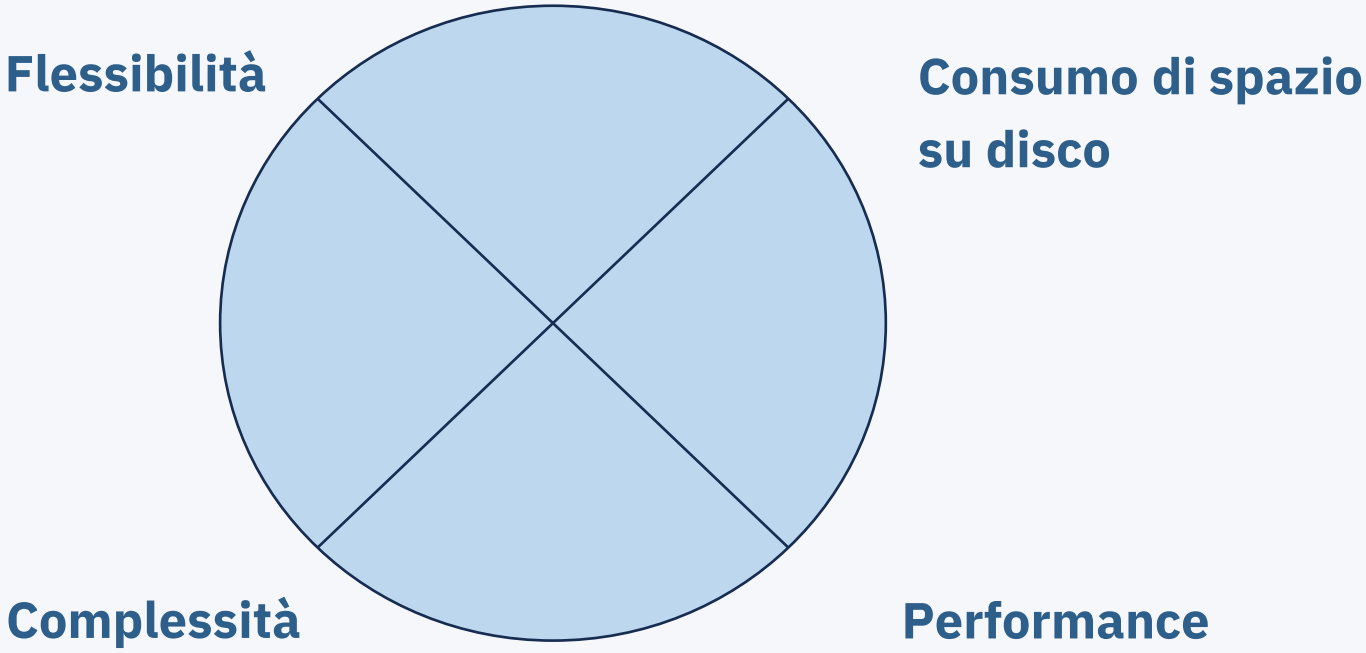
- Esiste un ordinamento fra le tuple che può essere rilevante ai fini della gestione

- **Organizzazione disordinata:**
  - Ordinamento fisico dei record ma non logico
  - Gli inserimenti vengono effettuati al posto di record cancellati, oppure in coda (con riorganizzazioni periodiche)
- **Organizzazione ordinata:**
  - L'ordinamento dei record è coerente con quello di un campo
  - Usata principalmente per memorizzare gli indici

Confronto

Ci sono molti modi per organizzare i record in memoria

Qual è il migliore?



LIVELLO FISICO DEL DBMS

Indici

INDICI

Indici

Indice

struttura ausiliaria per l'accesso (efficiente) ai record

accesso sulla base dei valori di uno o più campi

i campi sono detti **chiave** (ma non sono necessariamente identificanti)

Idea fondamentale: l'indice analitico di un libro

lista di coppie (termine, pagina), ordinata alfabeticamente sui termini, posta in fondo al libro e separabile da esso

INDICI

Tipi di indice

- **Indice primario:**
  - su un campo sul cui ordinamento è basata la memorizzazione
- **Indice secondario**
  - su una struttura non ordinata
  - su una struttura ordinata, ma su un campo che non determina l'ordinamento

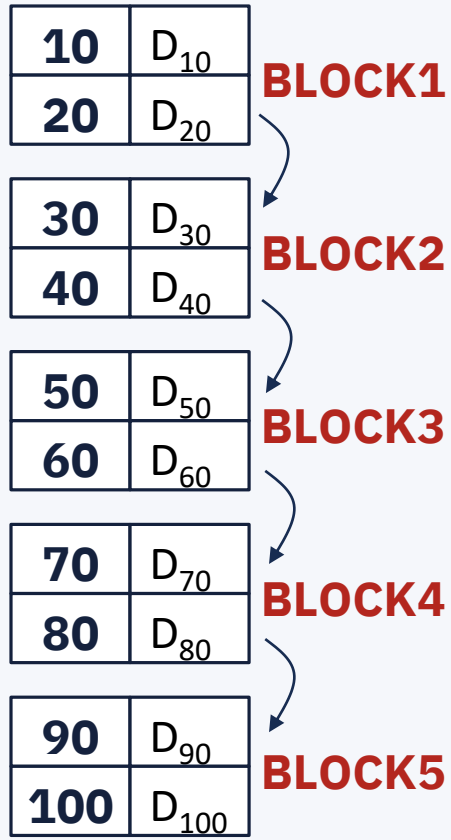
Tipi di indice

- **Indice denso:**
  - contiene una entry per ciascun valore del campo chiave
- **Indice sparso:**
  - contiene un numero di entry inferiore rispetto al numero di valori diversi del campo chiave

Indice denso vs sparso

D_x = Dati (altri campi) della riga con chiave x

Scrittura ordinata



Record su file su disco

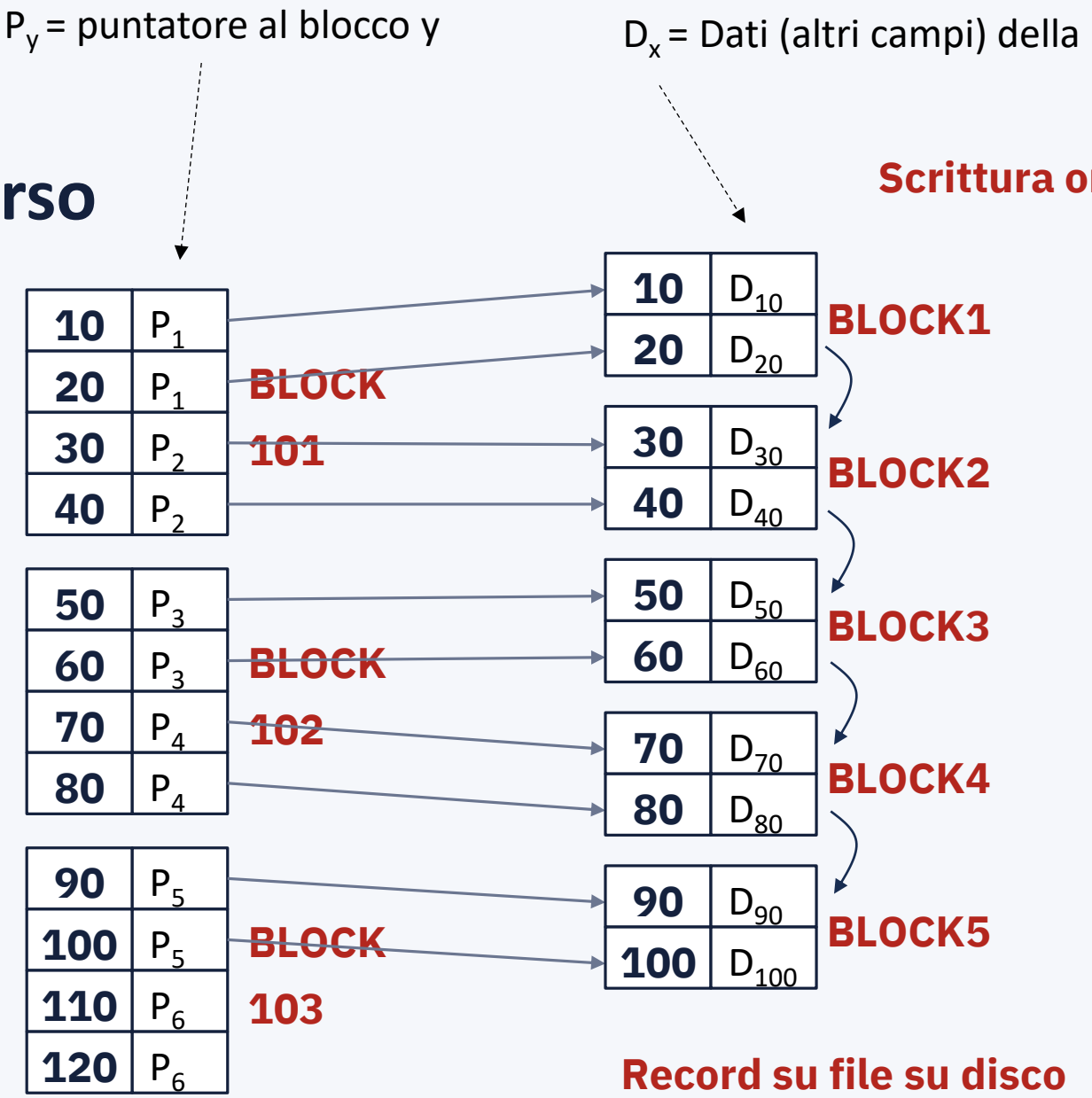
(2 record per blocco)

INDICI

Indice denso vs sparso

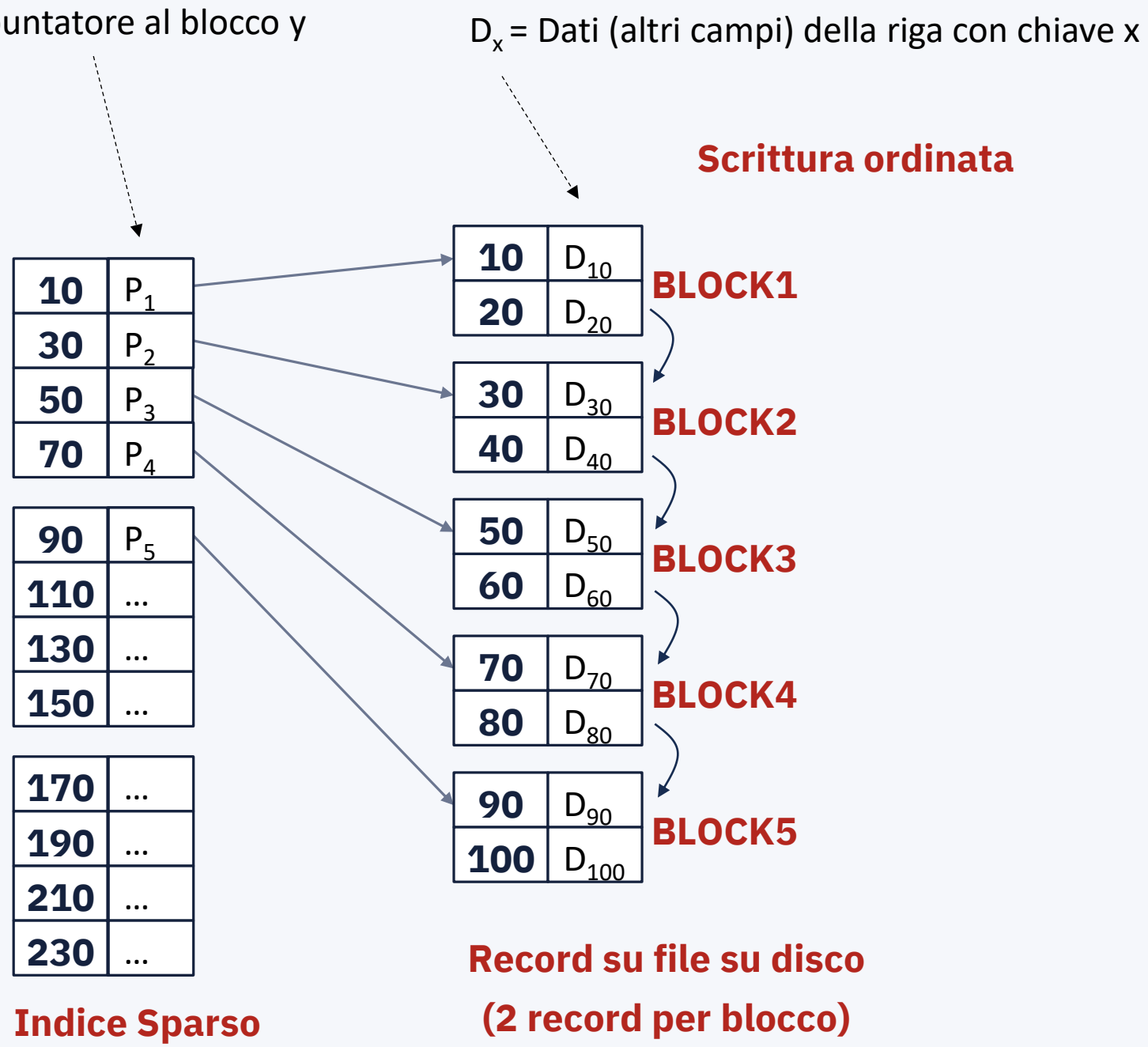
(Index / No index)
Vantaggi?
(pensa in termini di numero di blocchi da leggere)

Posso fare meglio?



Indice denso vs sparso

Posso fare meglio?



INDICI

Sparso vs. Denso: Tradeoff

Sparso: meno occupazione di spazio: posso tenere più parte dell'indice in RAM

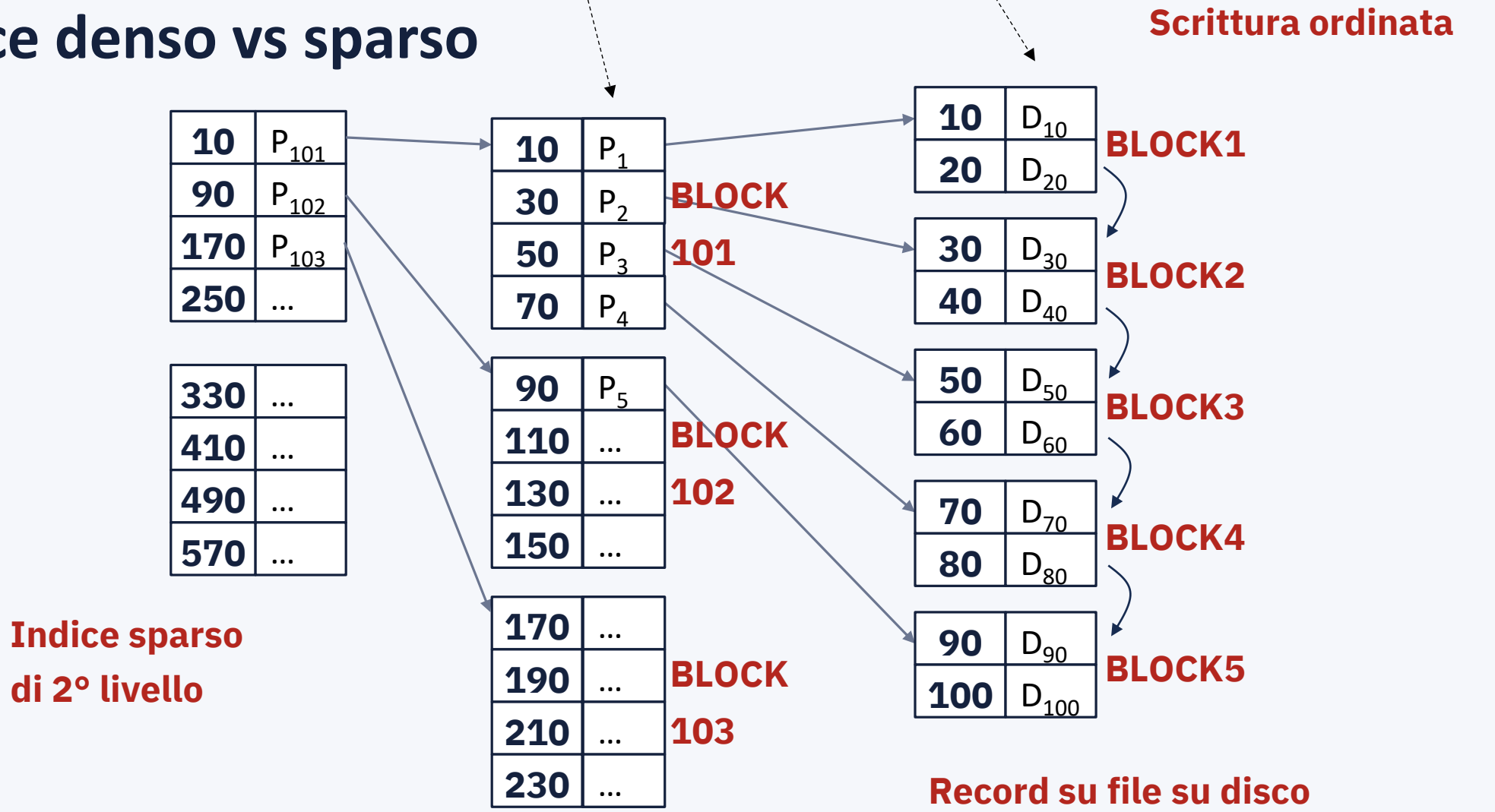
Denso: posso sapere se esiste un record con una certa chiave guardando solo l'indice

INDICI

Indice denso vs sparso

P_y = puntatore al blocco y

D_x = Dati (altri campi) della riga con chiave x



INDICI

Indici primari

Vantaggi

Semplici da implementare

Efficienti per scansioni (= letture di sequenze contigue di record)

Svantaggi

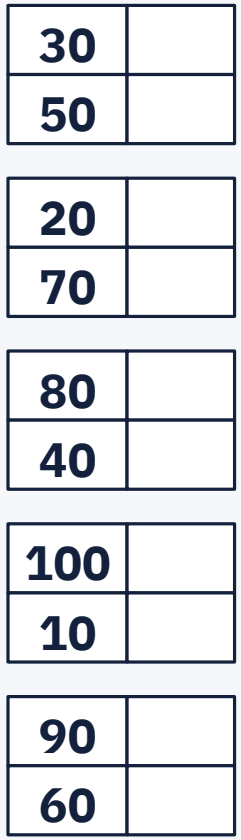
Modifiche costose

Difficile mantenere l'ordine

INDICI

Indici secondari

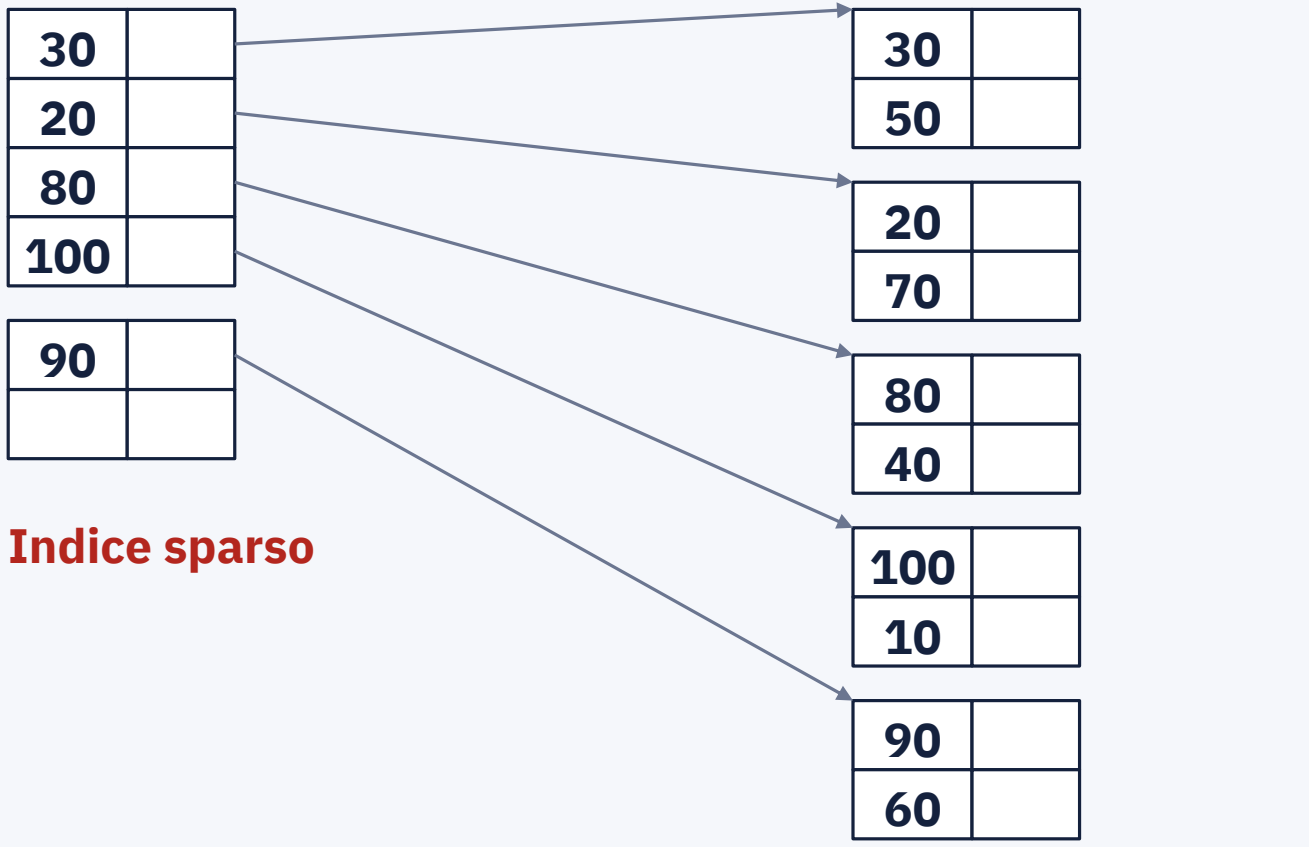
Come indicizzare una struttura disordinata?



Scrittura disordinata

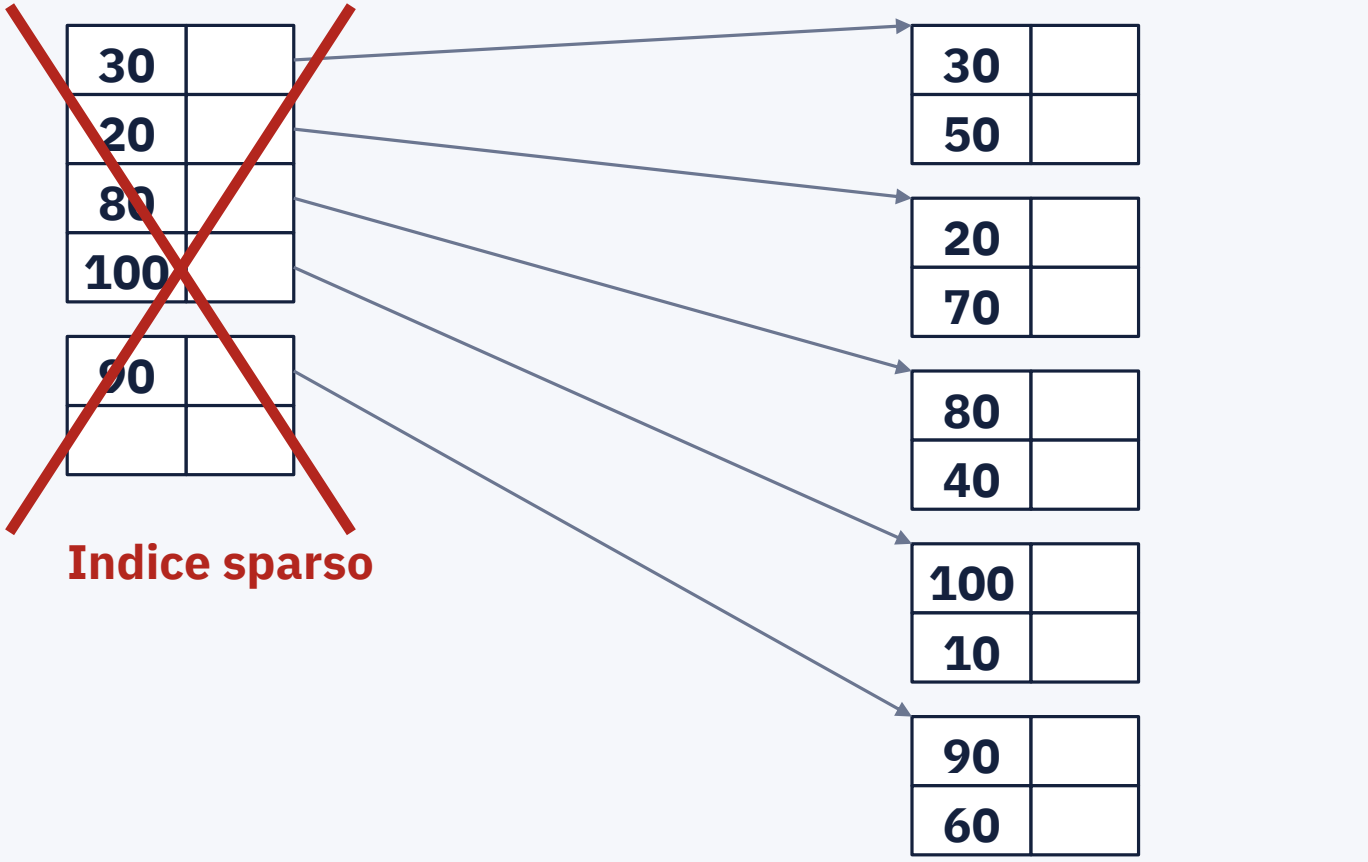
Indici secondari

Buona scelta?

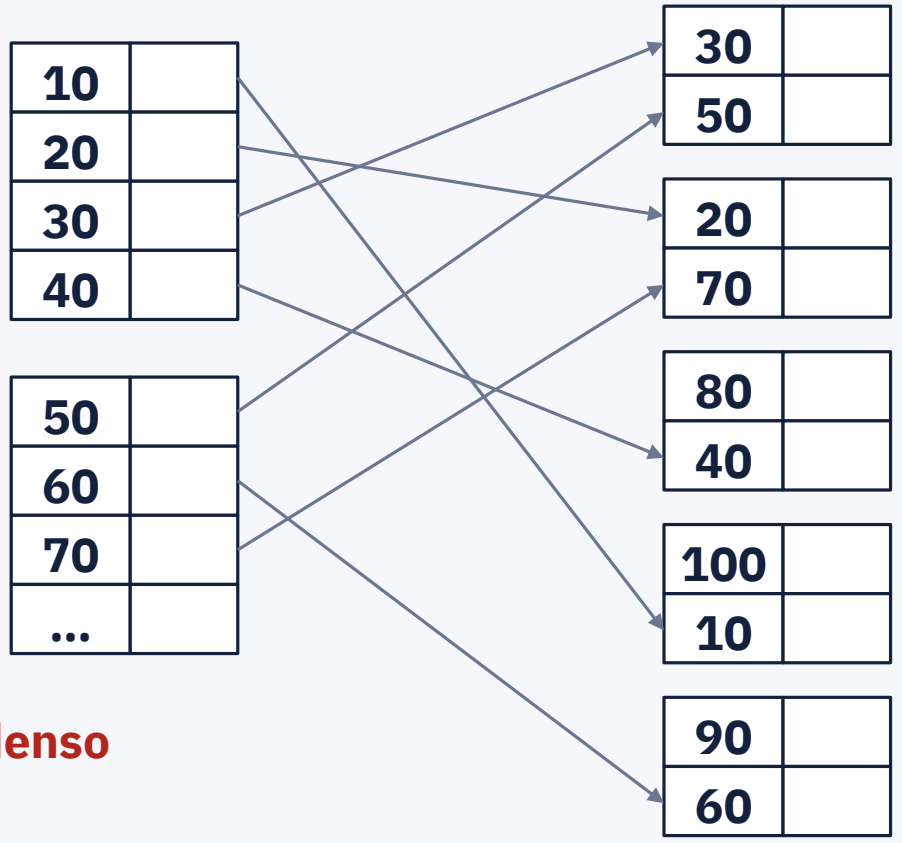


Indici secondari

Non ha senso...



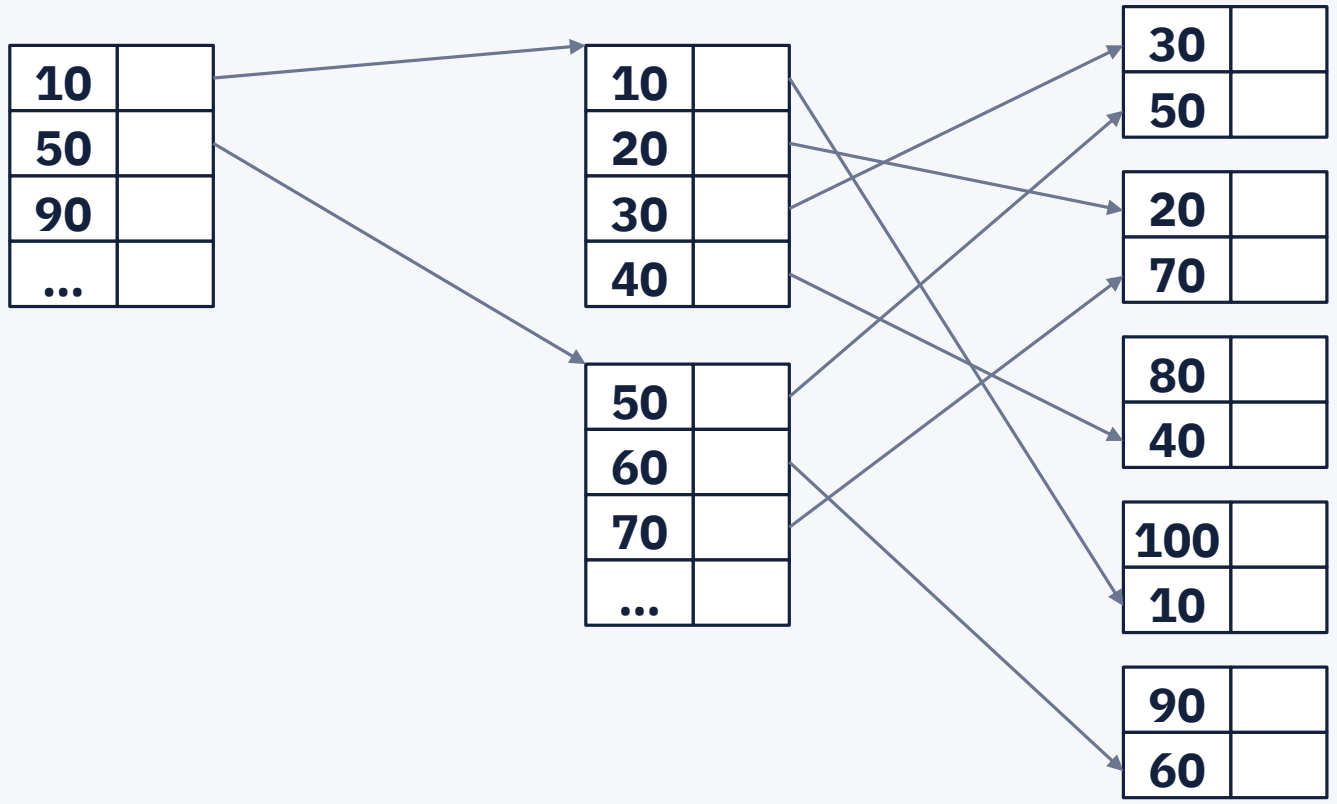
Indici secondari



Indice denso

Scrittura disordinata

Indici secondari



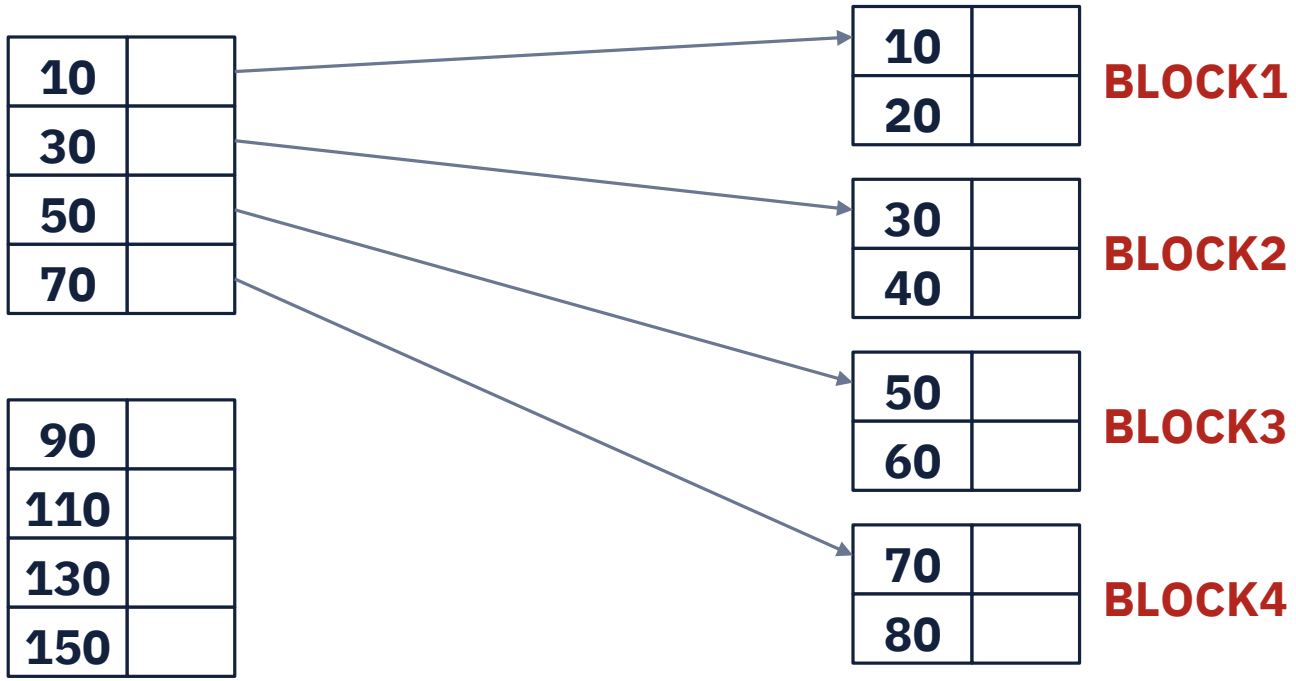
Indice sparso di 2° livello

Indice denso di 1° livello

Scrittura disordinata

Scrittura ordinata

Cancellazione da indice sparso

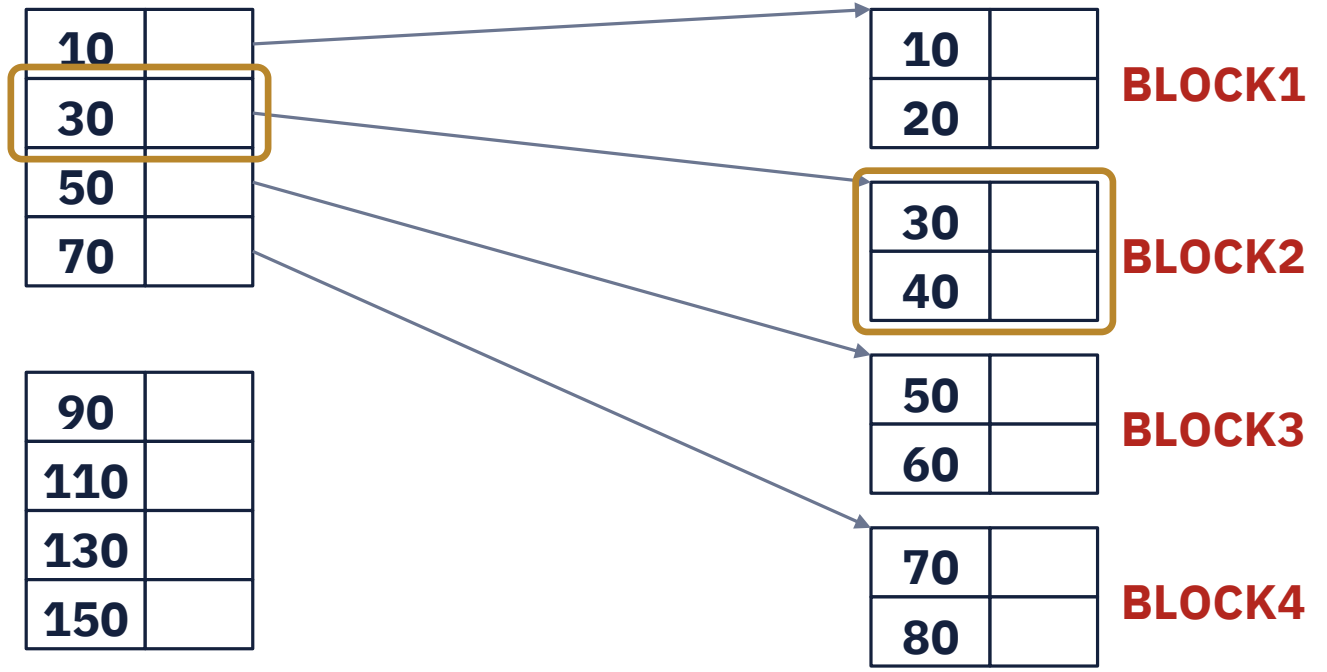


Record su file su disco

(2 record per blocco)

Cancellazione da indice sparso

– osserva BLOCK2 e la sua voce nell'indice



Nelle prossime slide: cosa succede a questo blocco e alla sua voce d'indice quando si cancellano i suoi record.

Cancellazione da indice sparso

Caso 1 — elimino il record 40 (non è la chiave)



L'indice non cambia: il record 40 non era la chiave del blocco (lo è 30).

Cancellazione da indice sparso

Caso 2 — elimino il record 30 (è la chiave)



L'indice si aggiorna: la chiave del blocco passa da 30 a 40, il primo record superstite.

Cancellazione da indice sparso

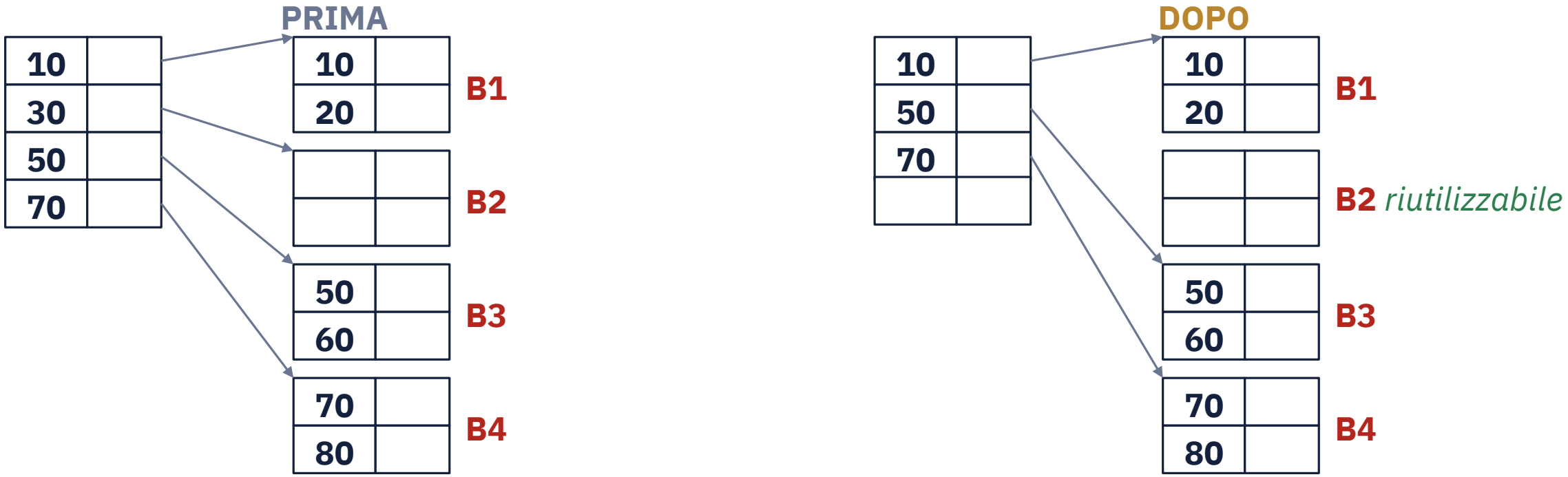
Caso 3 — elimino entrambi i record (30 e 40)



Il blocco è vuoto: l'indice non ha più bisogno di una voce per questo blocco.

Cancellazione da indice sparso

Caso 3 — effetto sull'indice completo



La voce scompare dall'indice: si passa direttamente da 10 a 50.

Cancellazione da indice sparso

Riepilogo: cosa cambia nel blocco e nell'indice

Caso	Nel blocco	Nell'indice
Record non chiave (es. 40)	Lo spazio si libera	Nessuna modifica: la chiave (30) è ancora valida
Record chiave (es. 30)	Il record viene eliminato	La chiave si aggiorna al primo record superstite (40)
Entrambi i record del blocco	Il blocco diventa vuoto e riutilizzabile	La voce dell'indice viene rimossa; i puntatori successivi si aggiornano

INDICI

Cancellazione da indice sparso

Prova tu

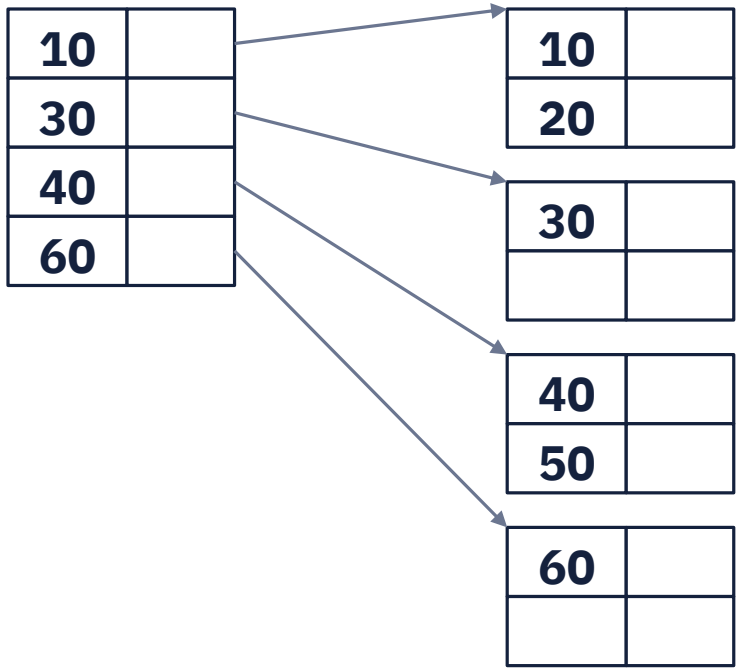
Cosa cambierebbe se cancellassi anche il record 50, chiave di BLOCK3?

E se, dopo aver svuotato BLOCK2, inserissi un nuovo record con chiave 35: dove andrebbe scritto?

Prossimo passo: l'inserimento in un indice sparso, quando nel blocco non c'è più spazio.

Inserimento in indice sparso

– **insert record 25**



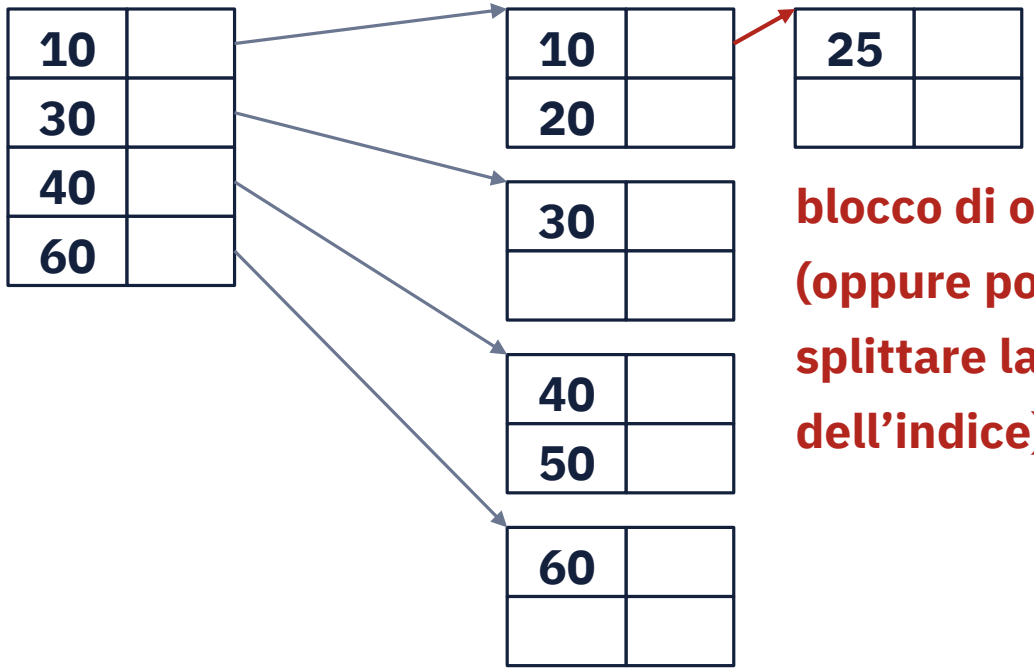
Cosa devo fare?

Cosa succede se non c'è spazio dopo il 30?

Scrittura ordinata

Inserimento in indice sparso

– insert record 25



blocco di overflow (oppure potrei splittare la radice dell'indice)

Svantaggi dei blocchi di overflow?

Scrittura ordinata

INDICI

Caratteristiche degli indici

Accesso diretto (sulla chiave) efficiente, sia puntuale sia per intervalli

E per le ricerche su altri campi (non chiave)?

Modifiche della chiave, inserimenti, eliminazioni inefficienti (come in tutte le strutture ordinate)

Tecniche per alleviare i problemi:

- blocchi di overflow
- marcatura per le eliminazioni
- riempimento parziale
- riorganizzazioni periodiche

INDICI

Indici, problemi

Il problema

Tutte le strutture di indice viste finora sono basate su strutture ordinate: sono quindi poco flessibili in presenza di elevata dinamicità

↓ *nei DBMS si usano indici ad-hoc, più sofisticati*

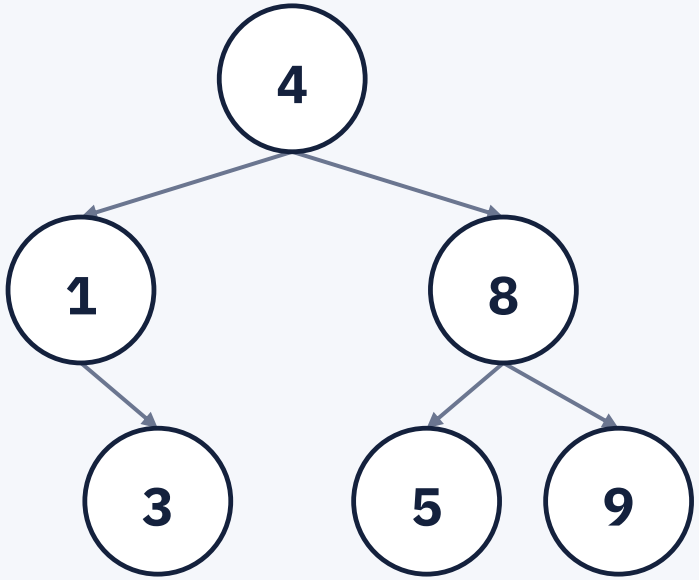
La soluzione: indici dinamici multilivello — **B-tree** (alberi di ricerca bilanciati)

ALBERI DI RICERCA

Albero binario di ricerca

Albero binario etichettato in cui, per ogni nodo, il sottoalbero sinistro contiene solo etichette minori di quella del nodo e il sottoalbero destro etichette maggiori

Tempo di ricerca e inserimento = profondità del nodo
logaritmico nel caso medio (assumendo un ordine di inserimento casuale)

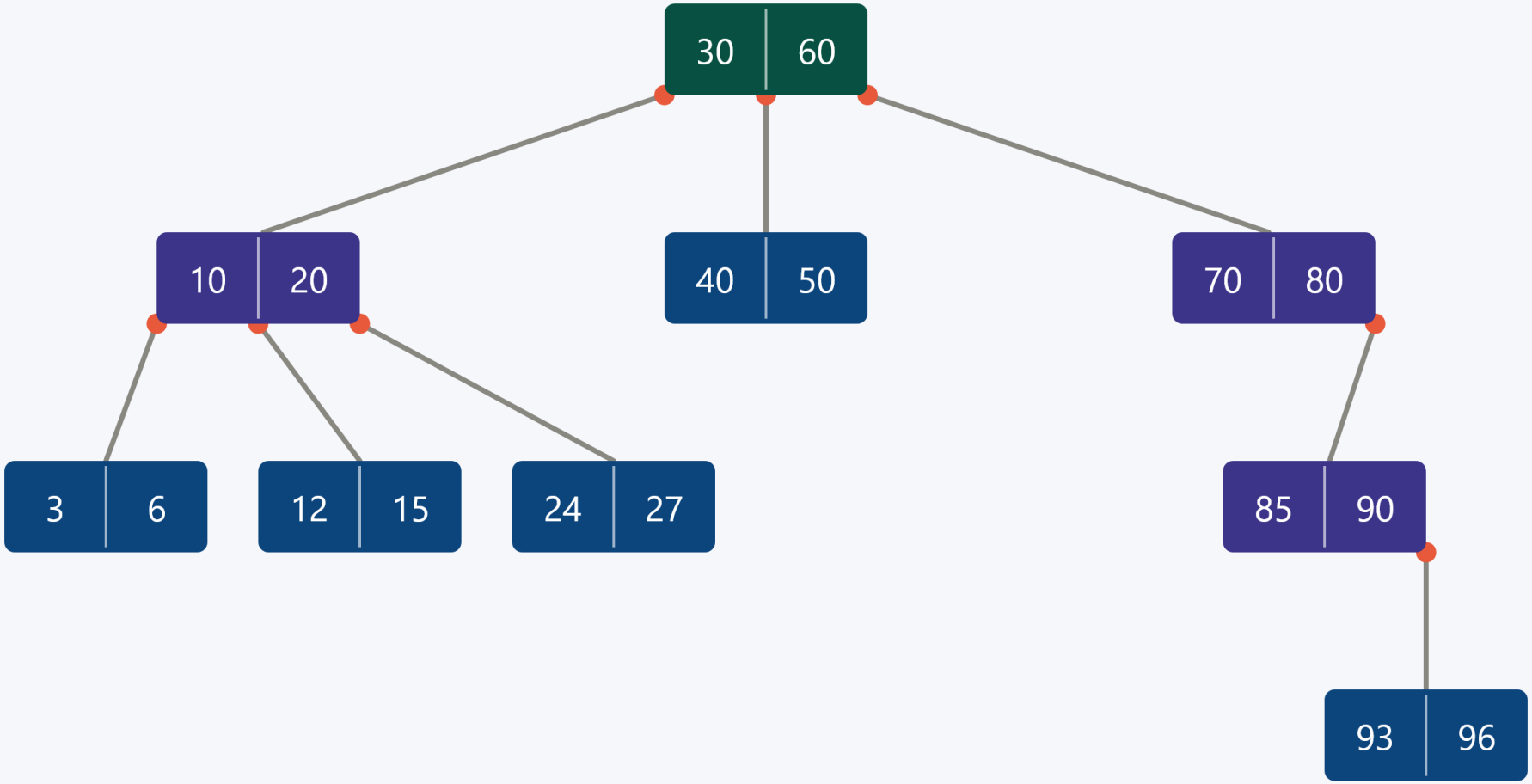


ALBERI DI RICERCA

Albero di ricerca di ordine P

1. Ogni nodo ha (fino a) P figli e (fino a) P–1 etichette, ordinate
2. Nell'i-esimo sottoalbero abbiamo tutte etichette maggiori della (i–1)-esima etichetta e minori della i-esima
3. Ogni ricerca o modifica comporta la visita di un cammino radice → foglia
4. In strutture fisiche, un nodo può corrispondere ad un blocco

Esempio: un albero di ricerca di ordine 3



ALBERI DI RICERCA

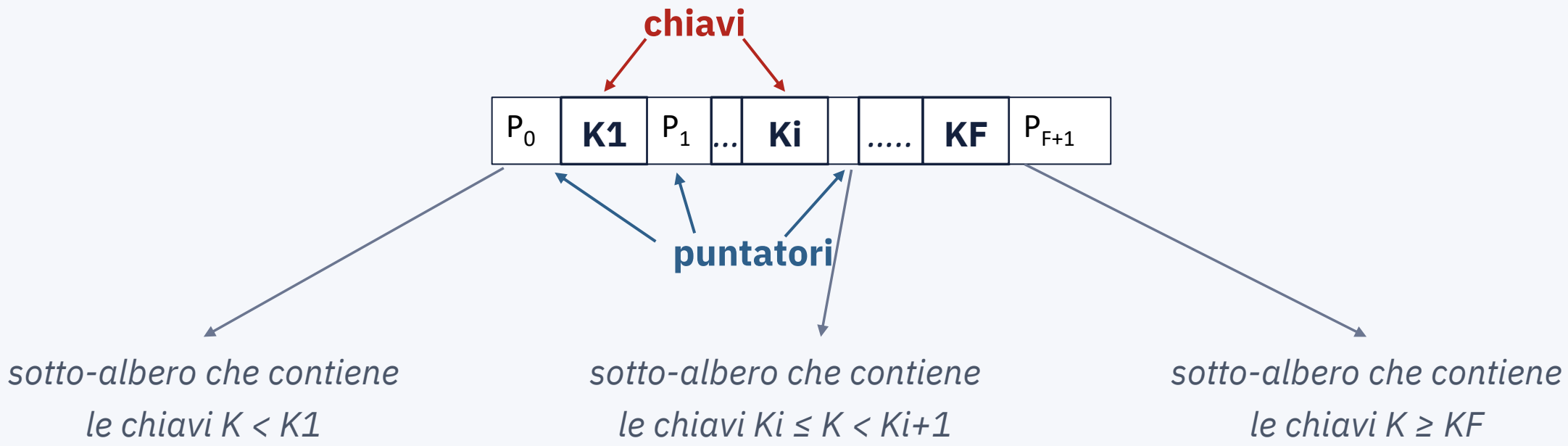
Albero di ricerca di ordine P

La struttura è ancora (potenzialmente) rigida

Un B-tree è un albero di ricerca mantenuto bilanciato grazie a:

1. **Riempimento parziale** (mediamente 70%)
2. **Riorganizzazioni locali** in caso di sbilanciamento

Organizzazione dei nodi del B-tree



ALBERI DI RICERCA

Split e merge

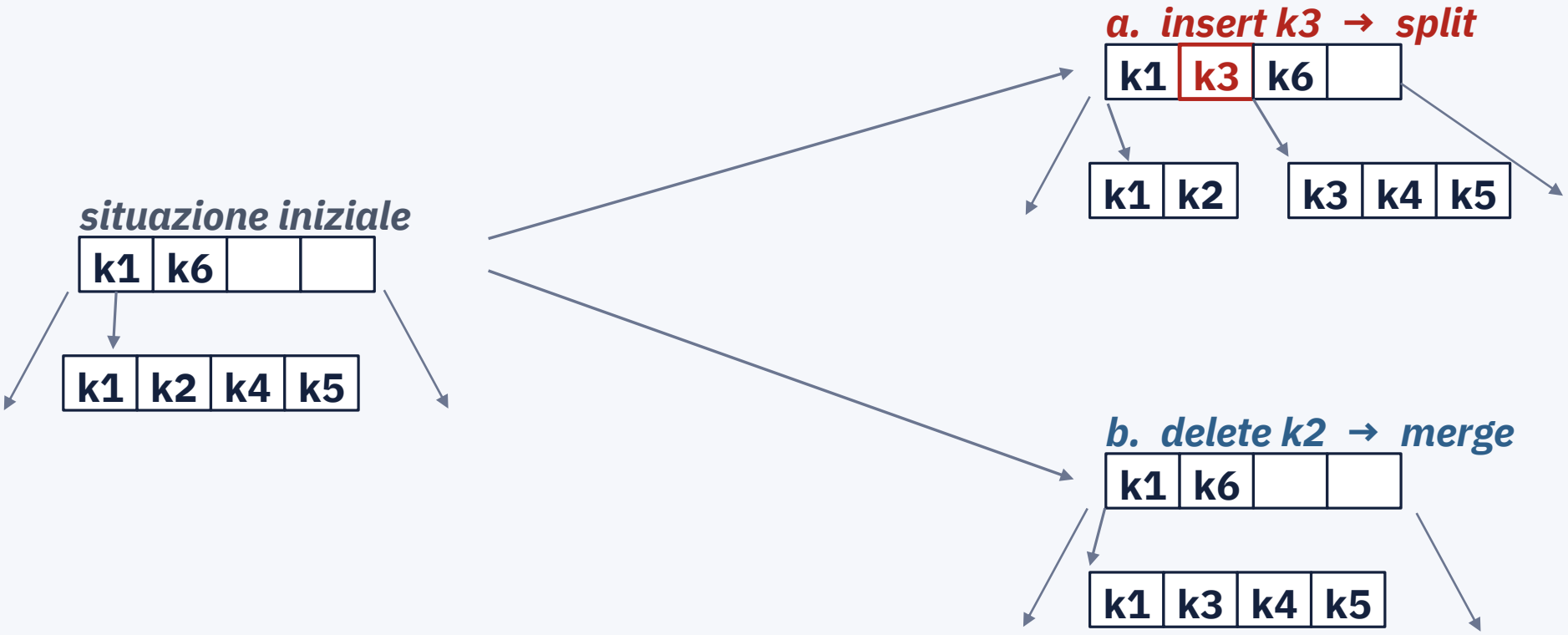
1. **Inserimenti ed eliminazioni** sono precedute da una ricerca fino ad una foglia

2. **Se c'è posto nella foglia**, si inserisce lì
3. **Altrimenti** il nodo va suddiviso (split), con un puntatore in più per il genitore

4. **Se non c'è posto neanche nel genitore**, si sale ancora nell'albero

Il riempimento rimane sempre superiore al 50%

Split e merge



ALBERI DI RICERCA

Split e merge — un esempio



La foglia piena si divide in due; la chiave mediana (25) sale al nodo genitore come nuovo separatore.

Esempio: https://www.youtube.com/watch?v=coRJrcIYbF4

ALBERI DI RICERCA

B tree e B+ tree

B tree

I nodi intermedi possono avere puntatori direttamente ai dati

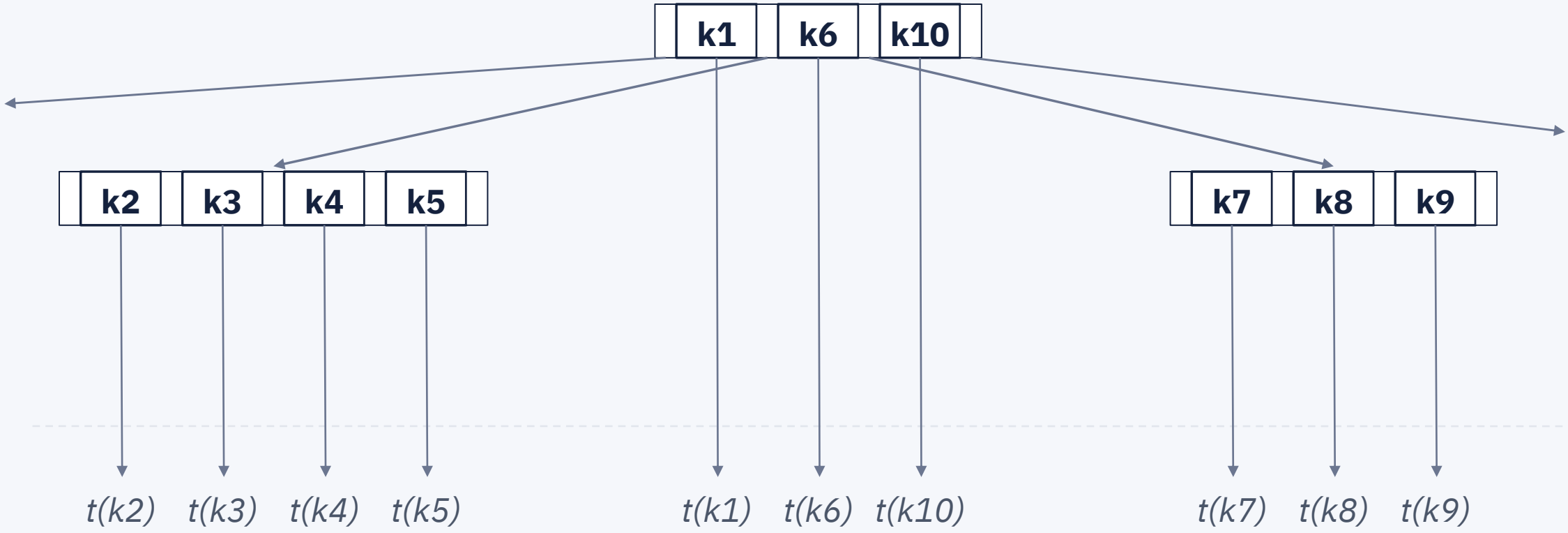
B+ tree

le foglie sono collegate in una lista

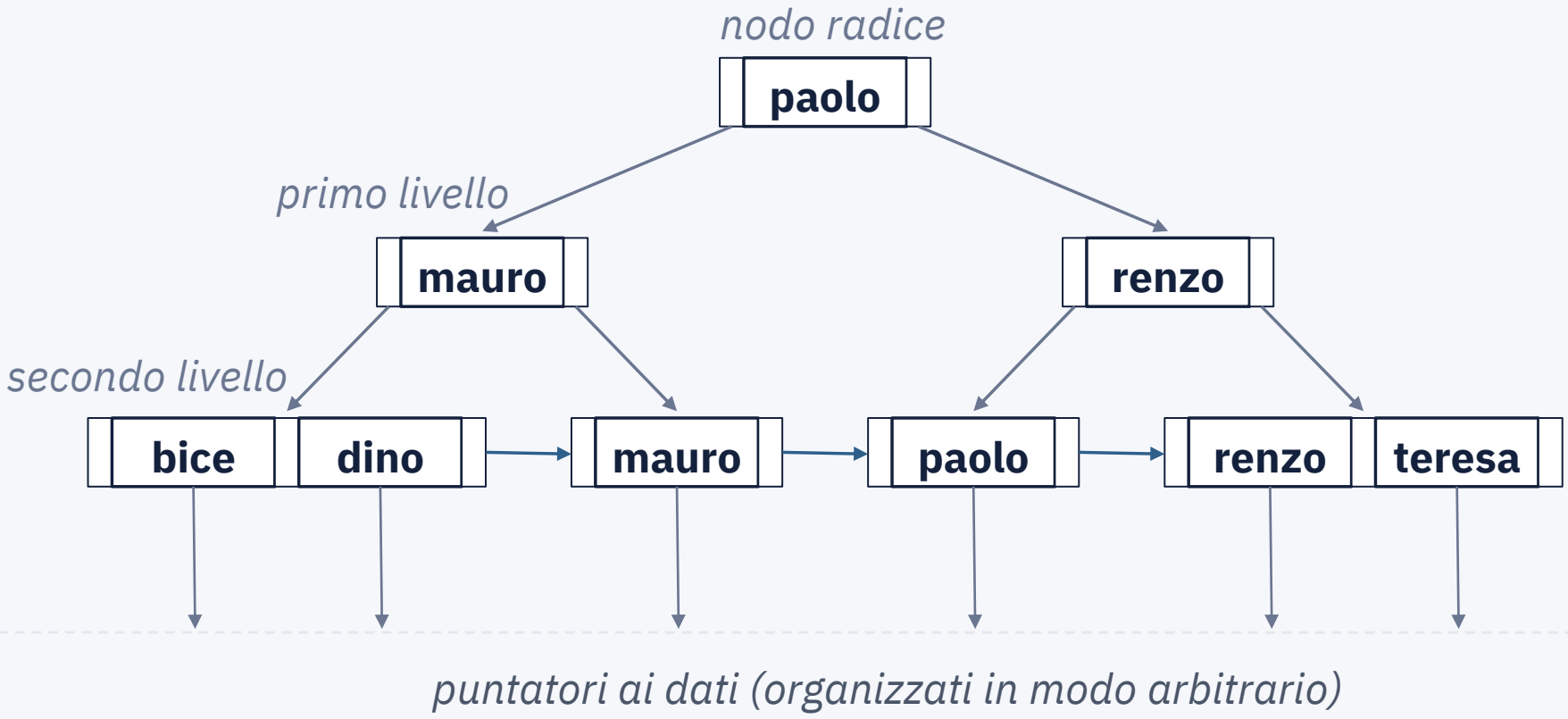
ottimi per le ricerche su intervalli

molto usati nei DBMS

Un B-tree


k1
k6
k10
k2
k3
k4
k5
k7
k8
k9
t(k2)
t(k3)
t(k4)
t(k5)
t(k1)
t(k6)
t(k10)
t(k7)
t(k8)
t(k9)

Un B+ tree


nodo radice
paolo
primo livello
mauro
renzo
secondo livello
bice
dino
mauro
paolo
renzo
teresa
puntatori ai dati (organizzati in modo arbitrario)

Efficienza dei B-Tree

Efficienza

- Molto efficienti quando il numero delle chiavi per blocco è grande
- Splitting e merging dei blocchi sono poco frequenti
  - e in genere limitati a piccole porzioni dell'albero

Ricerca

- Numero di letture = profondità dell'albero + 1 (per leggere il record su disco)
- Profondità tipica: 3 o 4 (eccetto DB molto grandi)

LIVELLO FISICO DEL DBMS

Hashing, processing, e ottimizzazione delle query

HASHING

Strutture ad accesso calcolato (hashing)

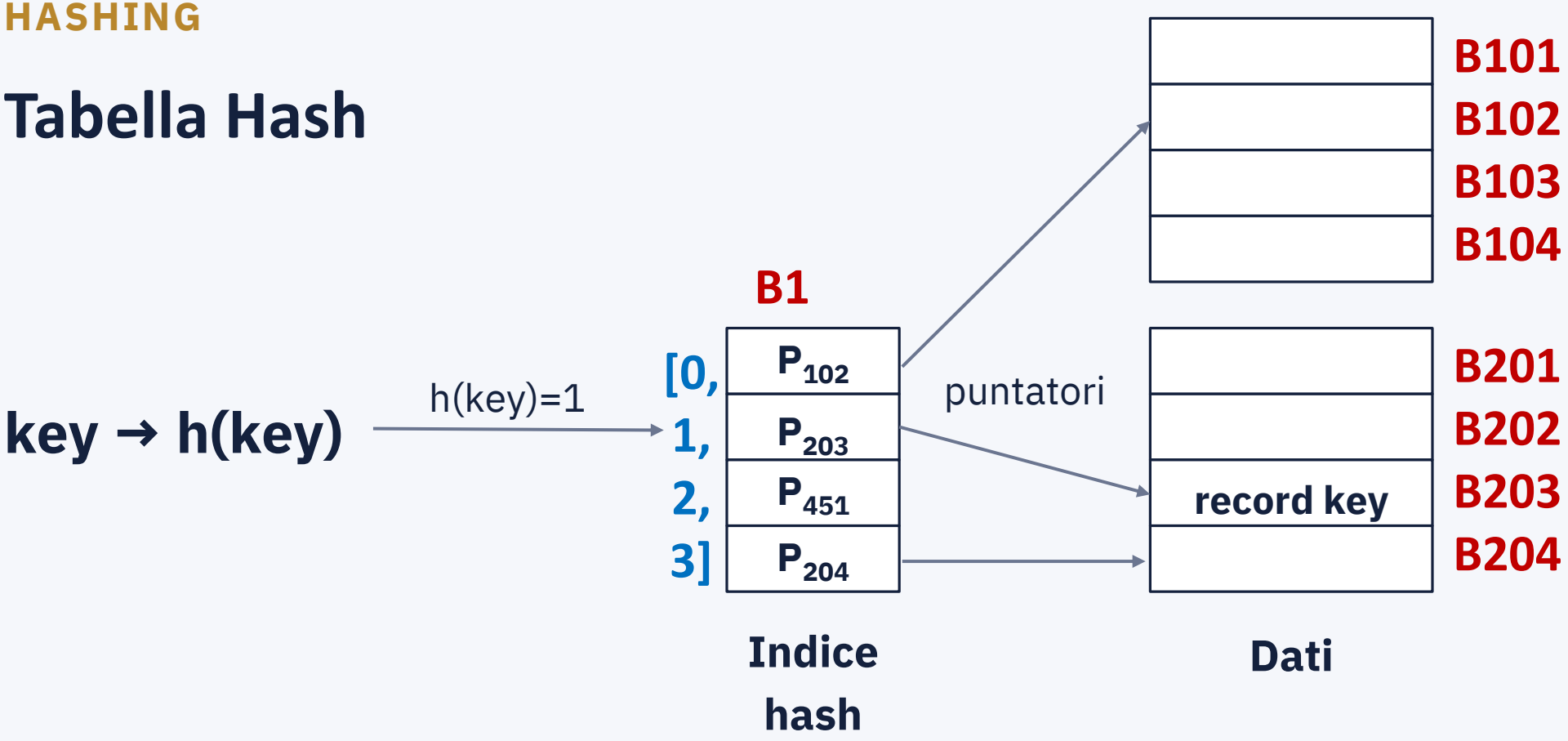
Obiettivo: accesso diretto ad un insieme di record sulla base del valore di un campo

1. **Il campo** è detto impropriamente “chiave”, anche se non è detto sia identificante

2. **La funzione hash** associa ad ogni valore della chiave un "indirizzo", in uno spazio di dimensione leggermente superiore al necessario

HASHING

Tabella Hash



HASHING

Funzione hash

Funzione h : C → I

- C = insieme delle possibili chiavi
- I: insieme degli indirizzi

Cardinalità(I) ≪ Cardinalità(C)

Una buona funzione hash deve distribuire i valori delle chiavi in I in modo uniforme

HASHING

Collisioni

Funzione hash:
il numero di possibili chiavi è molto maggiore del numero di possibili indirizzi

⇒ la funzione hash non può essere iniettiva: esistono **collisioni** — chiavi diverse che corrispondono allo stesso indirizzo

HASHING

Esempio di collisioni

Funzione hash h: `h(k) = k mod 50`

40 record, tavola hash con 50 posizioni

Riepilogo: 1 collisione a 4 · 2 collisioni a 3 · 5 collisioni a 2

M	M mod 50
60600	0
66301	1
205751	1
205802	2
200902	2
116202	2
200604	4
66005	5
116455	5
200205	5
201159	9
205610	10
201260	10
102360	10
205460	10
205912	12
205762	12
200464	14
205617	17
205667	17

M	M mod 50
200268	18
205619	19
210522	22
205724	24
205977	27
205478	28
200430	30
210533	33
205887	37
200138	38
102338	38
102690	40
115541	41
206092	42
205693	43
205845	45
200296	46
205796	46
200498	48
206049	49

Posizione 10 — 4 chiavi

205610 · 201260

102360 · 205460

2 posizioni — 3 chiavi ciascuna

pos 2: 205802 · 200902 · 116202

pos 5: 66005 · 116455 · 200205

5 posizioni — 2 chiavi ciascuna

pos 1: 66301 · 205751

pos 46: 200296 · 205796

(e altre 3 posizioni analoghe)

HASHING

Collisioni

Come ridurre la probabilità di collisione?

- Usando una “buona” funzione hash (che distribuisca in modo casuale e uniforme)
- Aumentando lo spazio ridondante

Come affrontare le collisioni?

- Posizioni successive disponibili
- Tabella di overflow

HASHING

File hash con fattore di blocco 10; 5 blocchi con 10 posizioni ciascuno

M mod 5	M mod 5	M mod 5	M mod 5	M mod 5
60600	66301	205802	200268	200604
66005	205751	200902	205478	201159
116455	115541	116202	210533	200464
200205	200296	205912	200138	205619
205610	205796	205762	102338	205724
201260		205617	205693	206049
102360		205667	200498	
205460		210522		
200430		205977		
102690		205887		
205845		206092		

40 record · tavola hash con 50 posizioni · file hash con fattore di blocco 10 · 5 blocchi con 10 posizioni ciascuno:

Due soli overflow

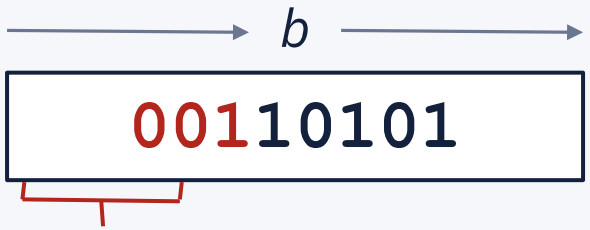
Come gestire la crescita di dimensione dei dati?

- Usare overflows e riorganizzazioni periodiche
- Hashing dinamico

Dynamic (extensible) hashing

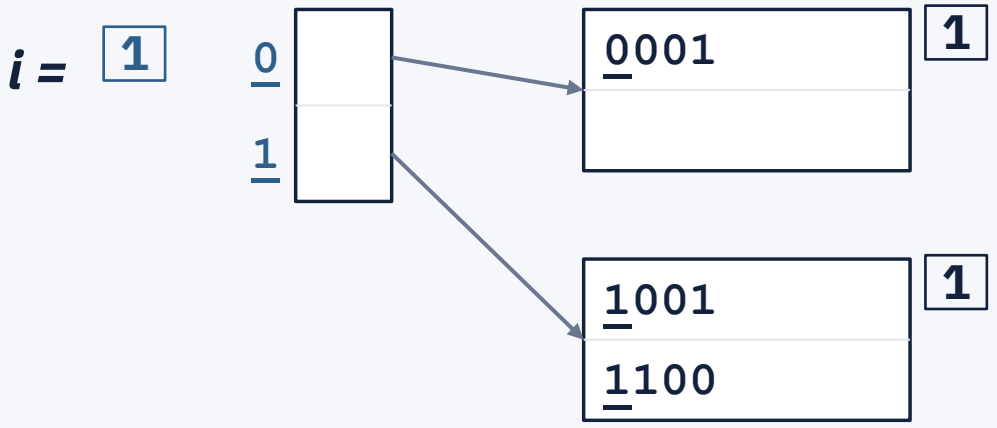
Invece di usare tutti i *b* bit di output della funzione di hash, usiamo solo i primi *i*

h(K) →



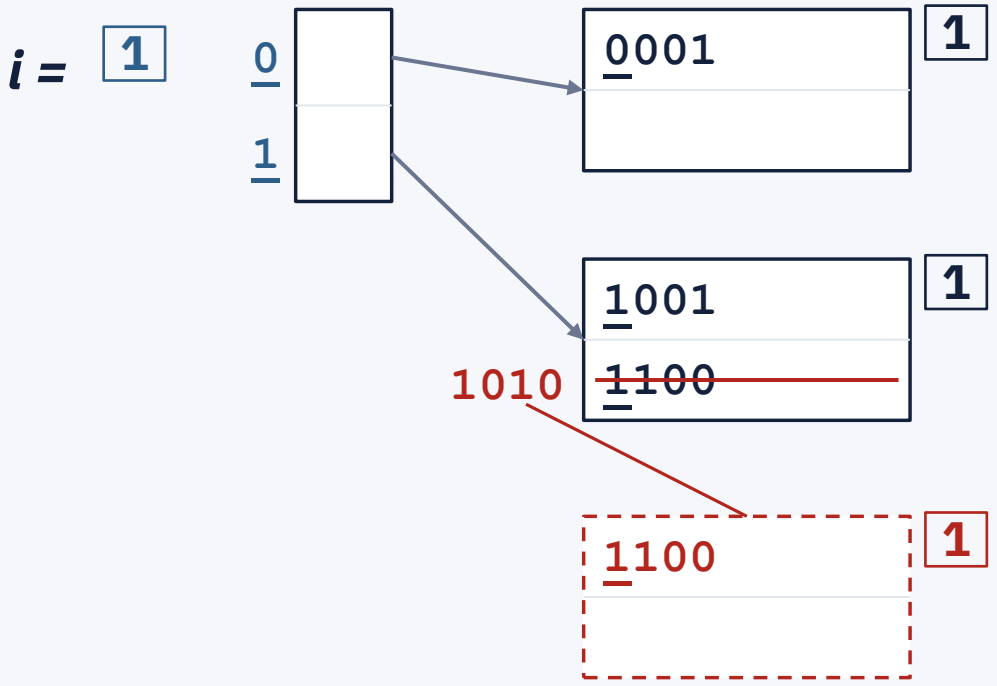
la dimensione di *i* cresce col tempo...

Esempio: h(k) restituisce 4 bit; 2 chiavi per blocco



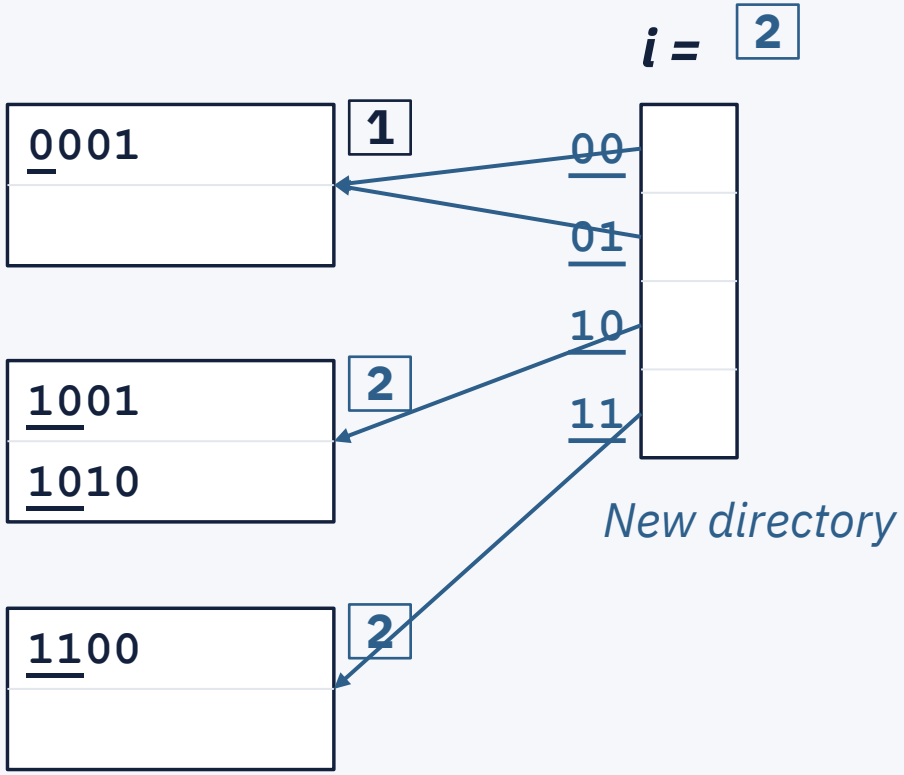
Insert: 1010

Esempio: h(k) restituisce 4 bit; 2 chiavi per blocco



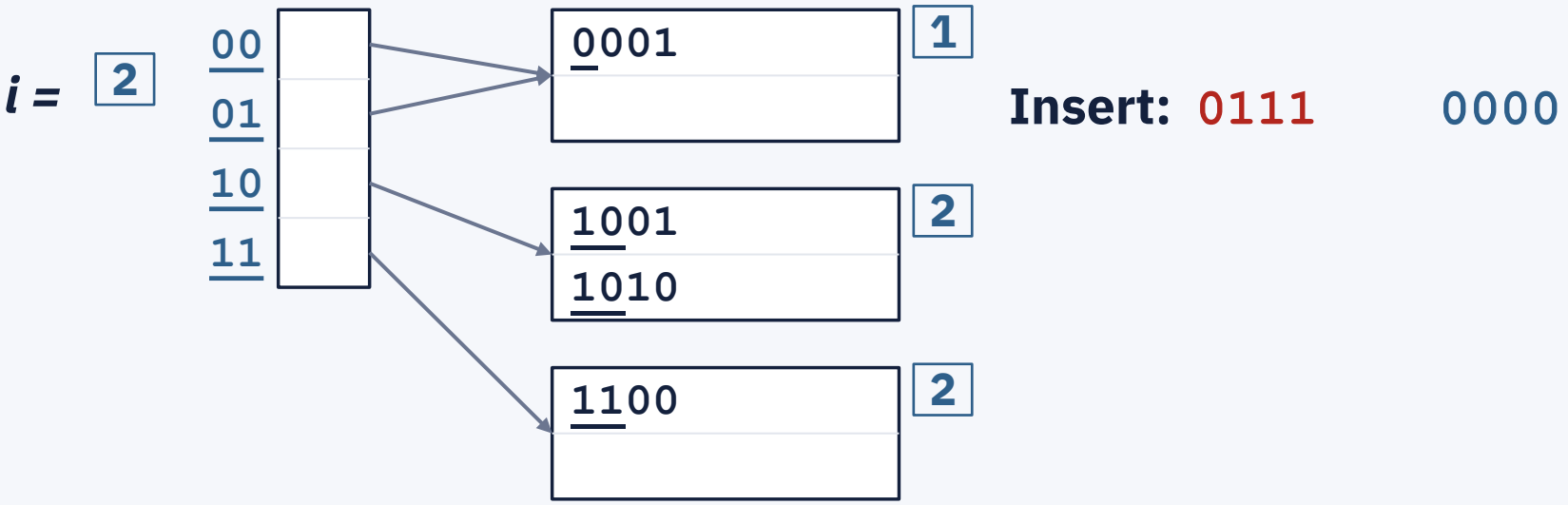
Insert: 1010

Esempio: h(k) restituisce 4 bit; 2 chiavi per blocco

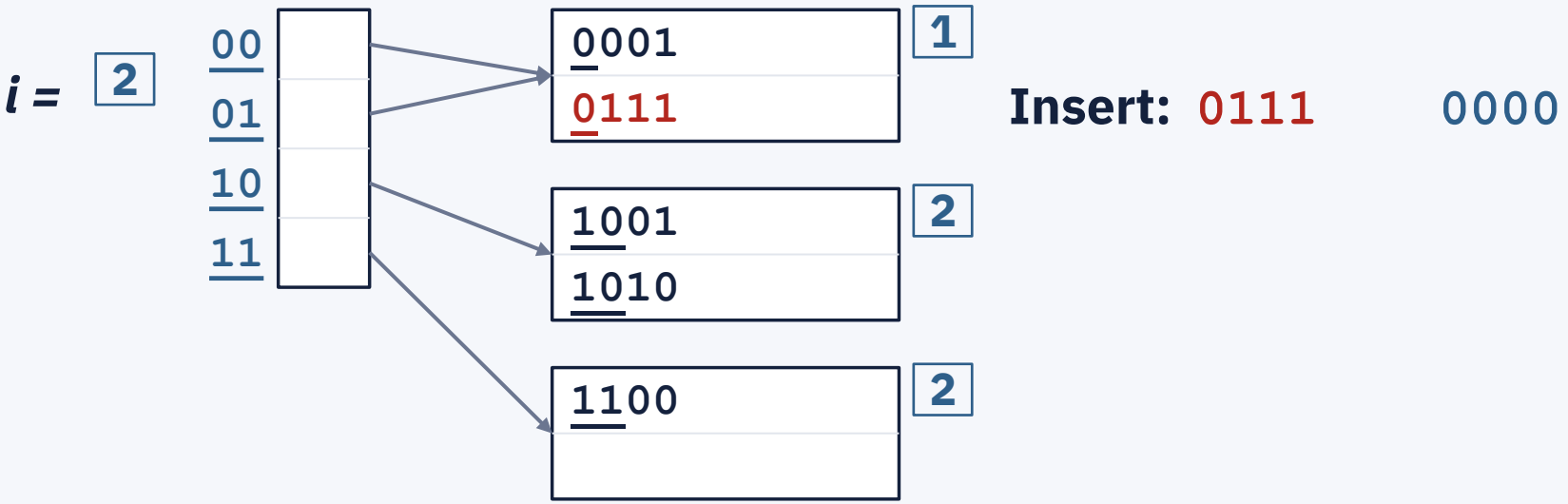


Insert: 1010

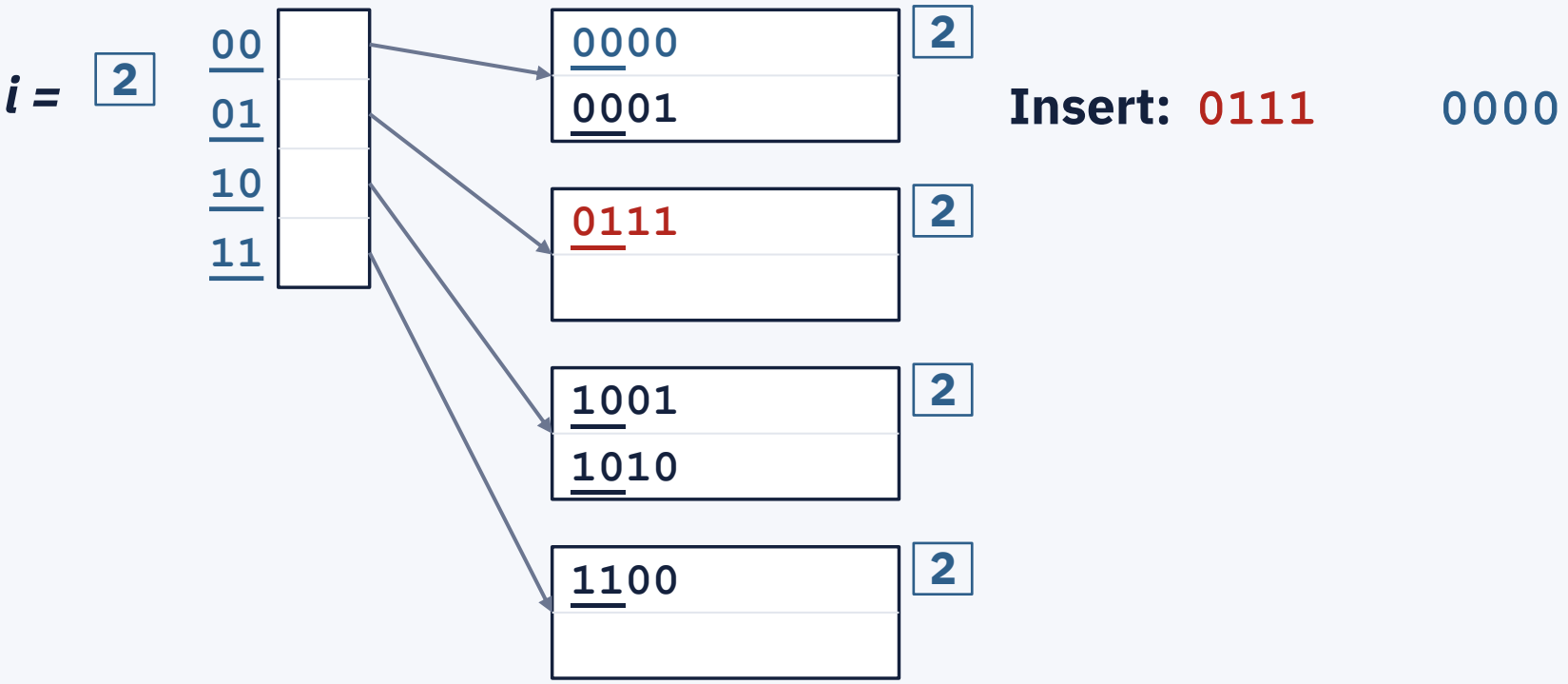
Esempio: h(k) restituisce 4 bit; 2 chiavi per blocco



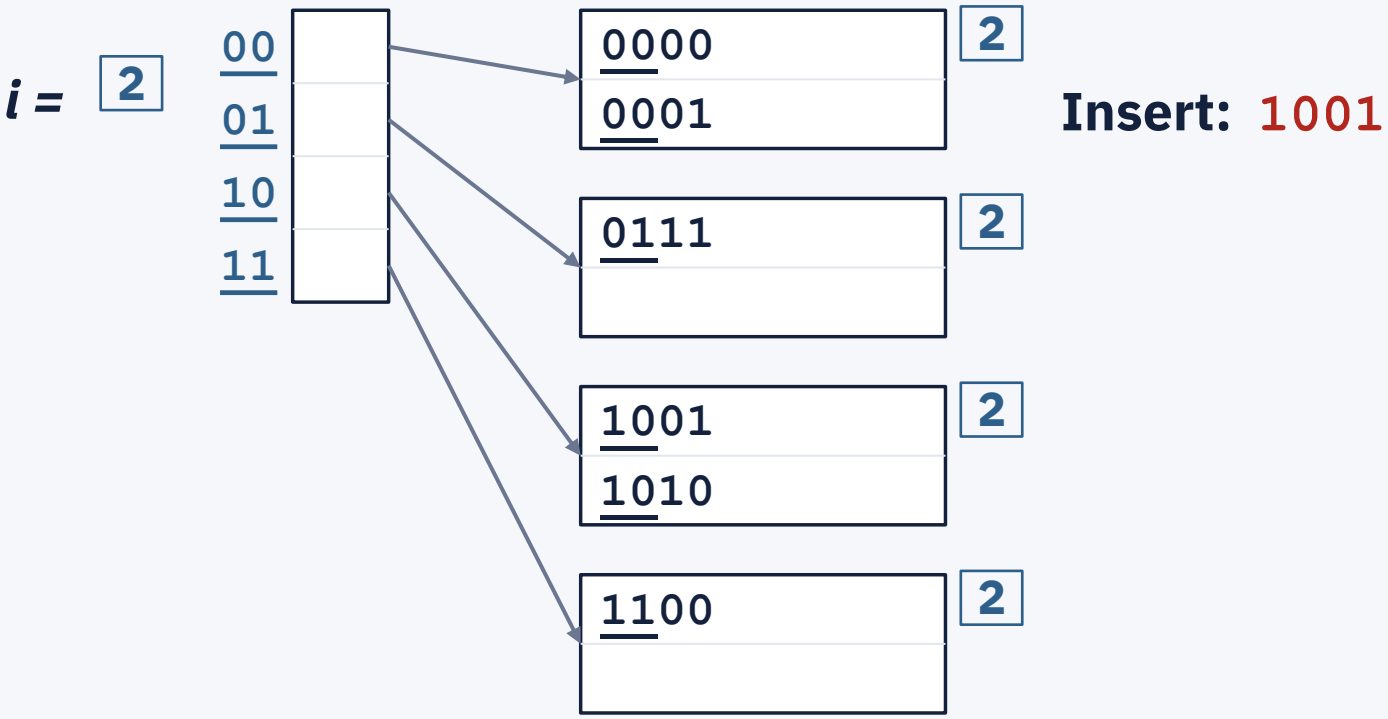
Esempio: h(k) restituisce 4 bit; 2 chiavi per blocco



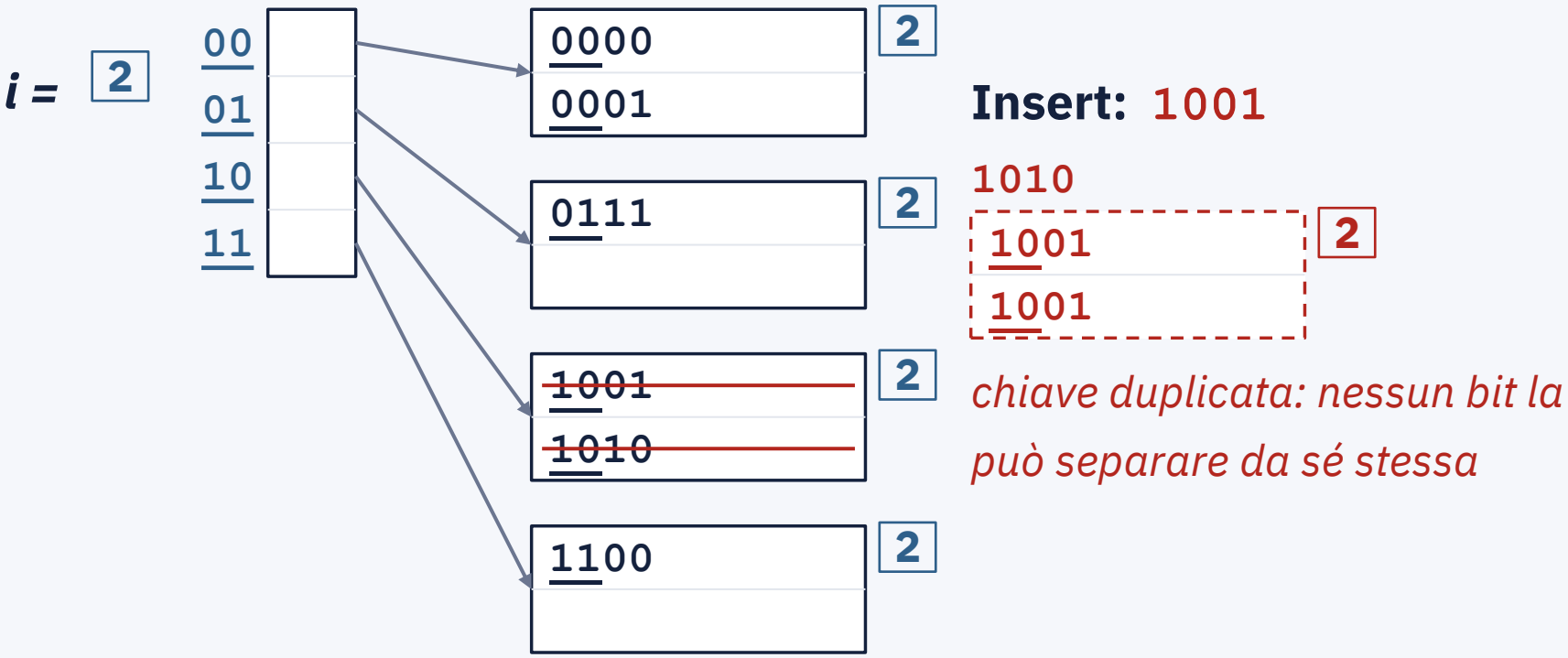
Esempio: h(k) restituisce 4 bit; 2 chiavi per blocco



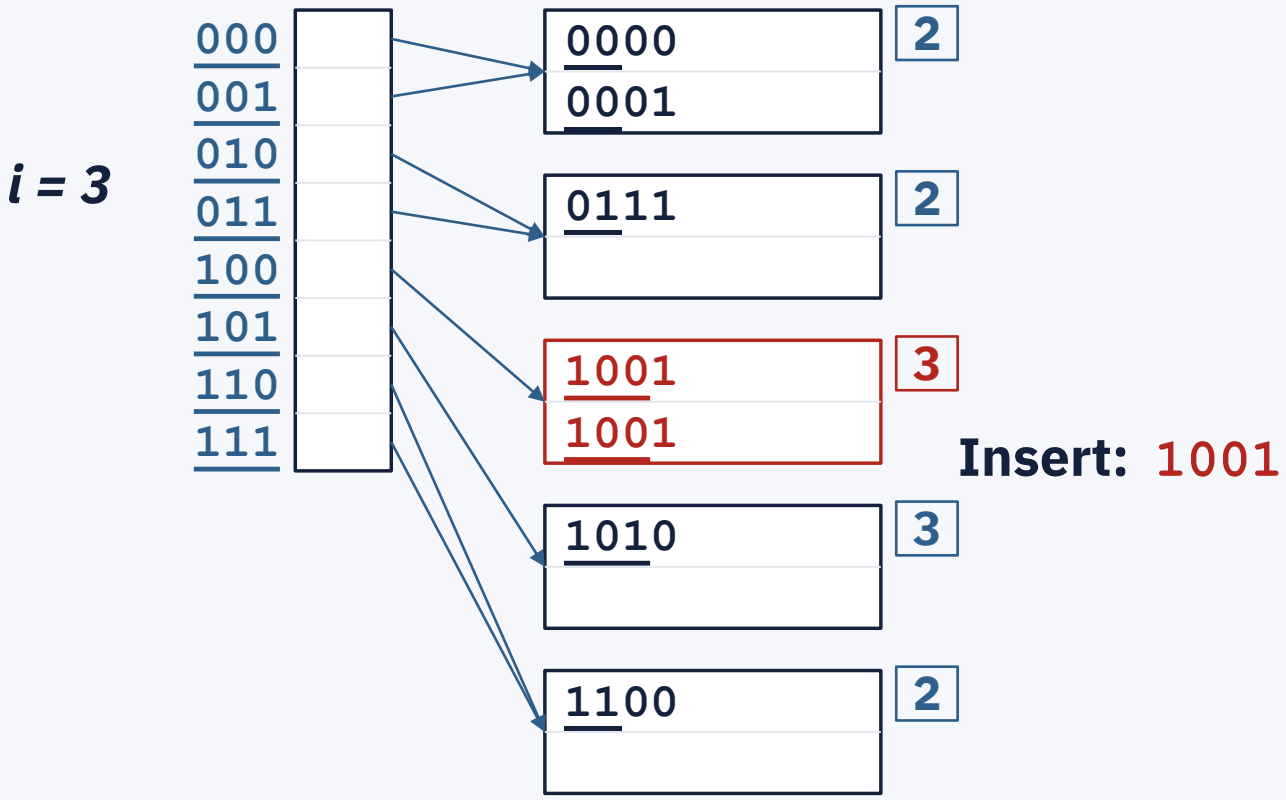
Esempio: h(k) restituisce 4 bit; 2 chiavi per blocco



Esempio: h(k) restituisce 4 bit; 2 chiavi per blocco



Esempio: h(k) restituisce 4 bit; 2 chiavi per blocco



HASHING

File hash, osservazioni

✓ **Organizzazione più efficiente** per l'accesso diretto con condizioni di uguaglianza

✗ Non efficiente per ricerche basate su intervalli

≈ Costo medio di poco superiore all'unità

! Il caso peggiore è molto costoso, ma improbabile

INDICI

Indice Bitmap

Adatto per colonne con un basso numero di possibili valori diversi

Stato civile	Single · Sposato
Range d'età	0-17 · 18-34 · 35-49 · 50-65 · 65+
Regione	Sardegna · Sicilia · Lombardia · ...

INDICI

Indice Bitmap

Tabella

Person_ID	Marital_Status
1001	Single
1002	Single
1003	Married
1004	Single
1005	Married
1006	Single
1007	Married
1008	Married
1009	Single

Indice Bitmap (Stato_Civile)

Single	Married
1	0
1	0
0	1
1	0
0	1
1	0
0	1
0	1
1	0

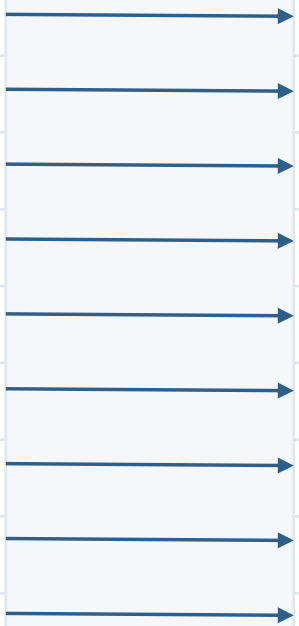
INDICI

Indice Bitmap

Numero di colonne = numero di valori differenti

Tabella

Person_ID	Age_Range
1001	18-34
1002	65+
1003	65+
1004	50-65
1005	35-49
1006	35-49
1007	65+
1008	35-49
1009	50-65



Indice Bitmap (Age_Range)

18-34	35-49	50-65	65+
1	0	0	0
0	0	0	1
0	0	0	1
0	0	1	0
0	1	0	0
0	1	0	0
0	0	0	1
0	1	0	0
0	0	1	0

INDICI

Indice Bitmap

"Trova tutti gli uomini anziani non sposati"

Tabella

Person_ID	Age_Range
1001	18-34
1002	65+
1003	65+
1004	50-65
1005	35-49
1006	35-49
1007	65+
1008	35-49
1009	50-65

Indice Bitmap (varie colonne)

18-34	65+	Married	Male
1	0	0	1
0	1	0	1
0	1	1	0
0	0	0	1
0	0	1	0
0	0	0	1
0	1	1	0
0	0	1	0
0	0	0	1

INDICI

Indice Bitmap

Pro

Efficiente per query su molti campi, ognuno con pochi valori possibili

Contro

Non adatto per dati molto dinamici (insert/update/delete frequenti)

una operazione su un dato può «bloccare» migliaia di altre operazioni sulla stessa tabella (scritture concorrenti sull'indice bitmap)

SQL

Definizione degli indici in SQL

Non è standard, ma presente nei vari DBMS

```
create [unique] index IndexName
  on TableName(AttributeList)
```

drop index IndexName

unique specifica che *AttributeList* è una superchiave

Esempio:

```
create index NomeIndice
  on Impiegato(Cognome, Citta)
```

Linee guida per gli indici

1. **Creare un indice** se si vuole recuperare *frequentemente* meno del *15%* delle righe di una *grande* tabella

2. **Indicizzare** le colonne usate per i join

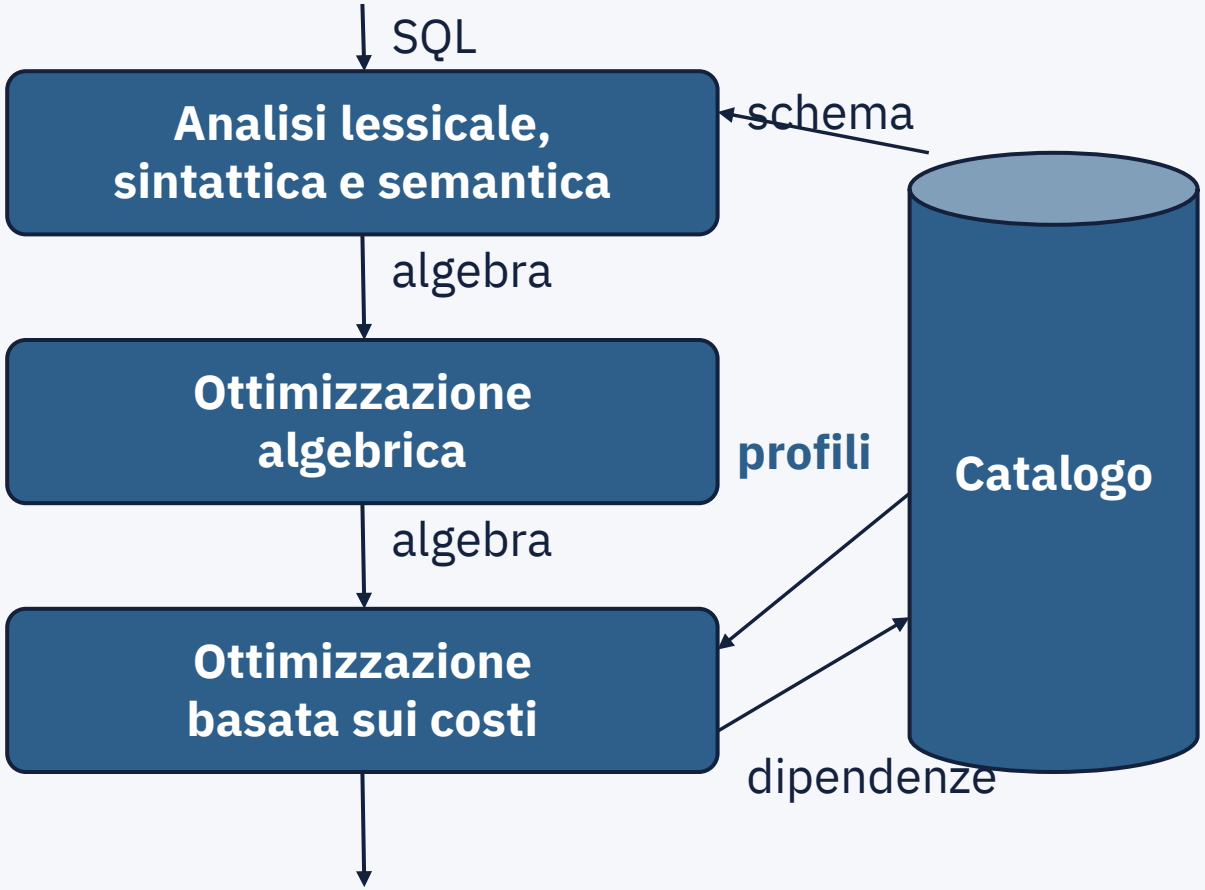
   (le chiavi primarie e i campi "unique" hanno automaticamente indici, ma può essere utile creare indici per le chiavi esterne)

3. **Indicizzare** le colonne i cui valori sono relativamente univoci (eccetto che per gli indici bitmap)

Esecuzione e ottimizzazione delle query

1. **Query processor** (o ottimizzatore): un modulo del DBMS
2. **Le query SQL sono espresse ad alto livello**
   - SQL è (di base) un linguaggio dichiarativo
   - L'algebra relazionale offre un approccio procedurale
3. **L'ottimizzatore** sceglie la strategia realizzativa a partire dall'istruzione SQL

Esecuzione delle query



OTTIMIZZAZIONE DELLE QUERY

Ottimizzazione algebrica

Il termine *ottimizzazione* è improprio perché il processo utilizza euristiche

Si basa sulla nozione di equivalenza:
due espressioni sono equivalenti se producono lo stesso risultato qualunque sia l'istanza attuale della base di dati

I DBMS cercano di eseguire espressioni equivalenti a quelle date, ma meno "costose"

OTTIMIZZAZIONE DELLE QUERY

Ottimizzazione algebrica

Euristica fondamentale:

selezioni e proiezioni il più presto possibile (per ridurre le dimensioni dei risultati intermedi):

```
"push selections down"
```

```
"push projections down"
```

"Push selections down"

Assumiamo A attributo di R2

```
SEL A=10 (R1 JOIN R2)
```

⇩

```
R1 JOIN (SEL A=10 (R2))
```

La seconda forma riduce in modo significativo la dimensione del risultato intermedio

(e quindi il costo dell'operazione)

Rappresentazione interna delle query

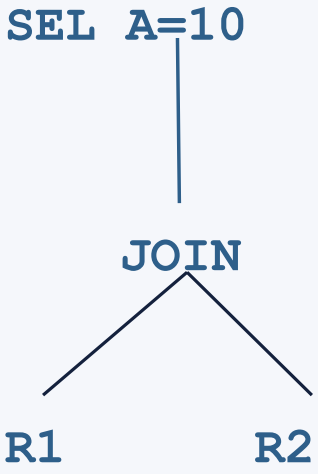
Tramite alberi:

1. **Foglie:** dati
2. **Nodi intermedi:** operatori

   (operatori algebrici, poi effettivi operatori di accesso)

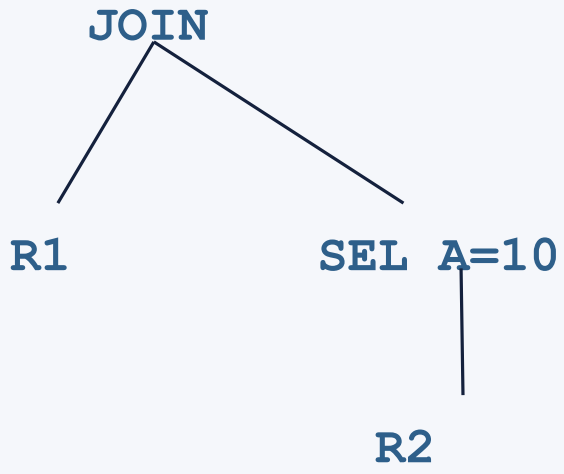
Alberi per la rappresentazione di query

```
SEL A=10 (R1 JOIN R2)
```



Più efficiente →

```
R1 JOIN (SEL A=10 (R2))
```



ESECUZIONE DELLE QUERY

Esecuzione delle operazioni

I DBMS implementano gli operatori dell'algebra per mezzo di operazioni di basso livello, *che possono implementare vari operatori "in un colpo solo"*

Operatori fondamentali

Scansione e accesso diretto

A livello più alto

Ordinamento e join

ESECUZIONE DELLE QUERY

Accesso diretto e scansione

Accesso diretto possibile solo se uso:

- Indici (B-tree...)
- Strutture hash

Scansione efficiente se uso:

- B+ tree

JOIN

Join

`R1 JOIN R2`

L'operazione più costosa (e molto frequente)

Vari metodi; i più noti:

1. **Nested-loop** (chiamato anche "iteration join")
2. **Merge-join** (chiamato anche "merge scan")
3. **Hash-join**

JOIN

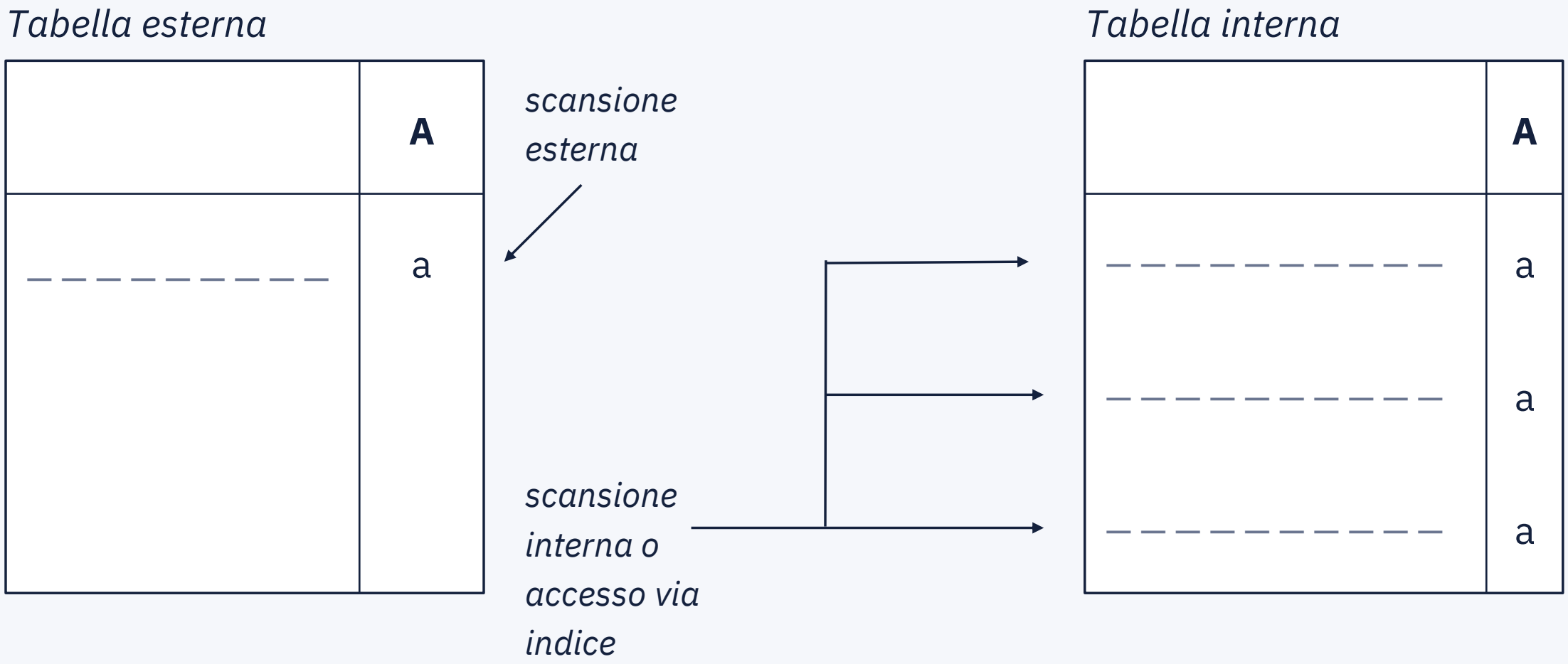
1) Nested-loop

- (concettualmente) R1 JOIN R2 ON R1.C = R2.C:

```
for each r ∈ R1 do
    for each s ∈ R2 do
        if r.C = s.C then output <r,s> pair
```

JOIN

1) Nested-loop



JOIN

2) Merge-join

- (concettualmente) R1 JOIN R2 ON R1.C = R2.C:

  (1) Ordinare R1 e R2 su C, o indicizzarli con B+ tree

  (2) i ← 1; j ← 1;

  while (i ≤ #T(R1)) ∧ (j ≤ #T(R2)) do

  if R1{i}.C = R2{j}.C then { outputTuples(R1{i}.C) ; i ← i+1 ; j ← j+1 }

  else if R1{i}.C > R2{j}.C then { j ← j+1 }

  else if R1{i}.C < R2{j}.C then { i ← i+1 }

JOIN

2) Merge-join

Tabella sinistra

	A
	a
	b
	b
	c
	c
	e
	f
	h

scan sinistro

scan destro

Tabella destra

	A
	a
	a
	b
	c
	e
	e
	g
	h

JOIN

3) Hash-join

```
Hash function h, range 0 → k
Buckets per R1: G0, G1, ... Gk     ·     Buckets per R2: H0, H1, ... Hk
```

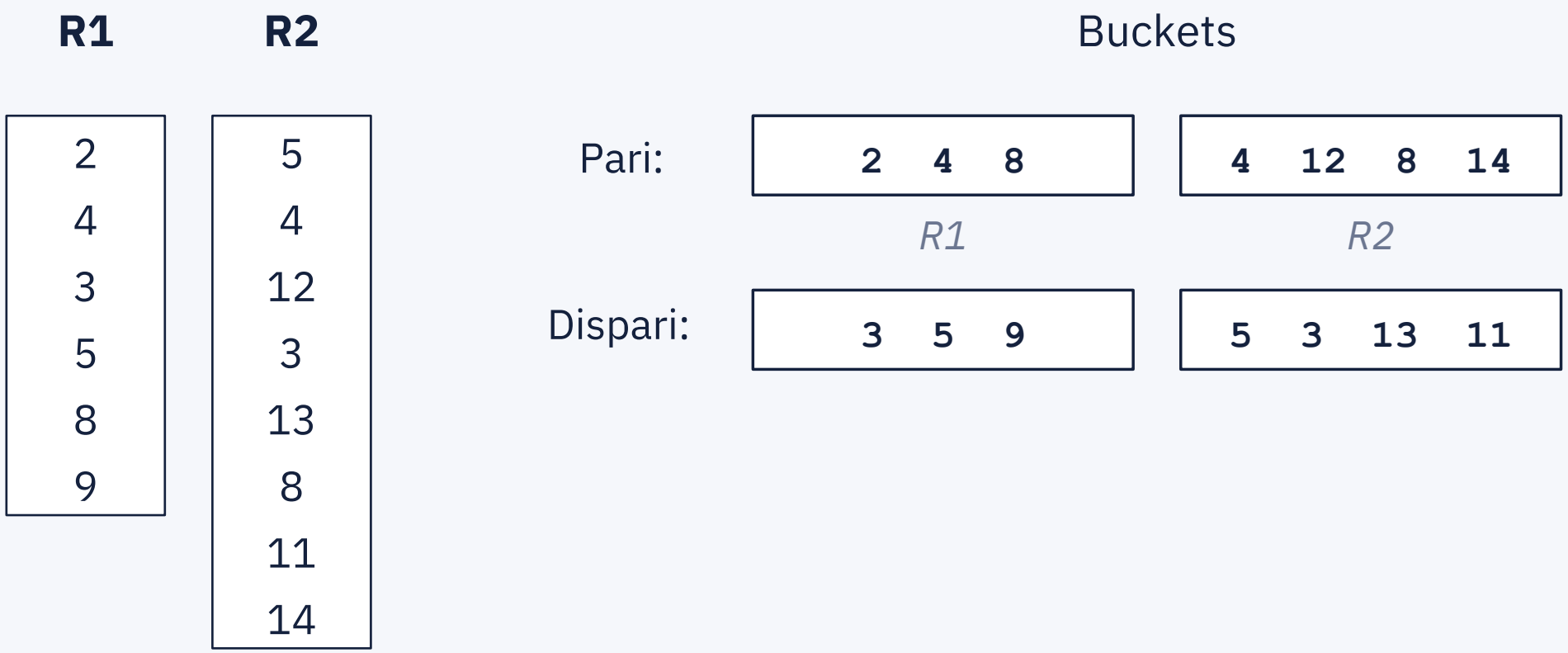
Algoritmo per R1 JOIN R2 ON R1.C = R2.C:

1. Fare hash delle tuple di R1 in G buckets (in base al valore di C)
2. Fare hash delle tuple di R2 in H buckets (in base al valore di C)
3. For i = 0 to k do: confrontare le tuple nei bucket Gi e Hi

JOIN

3) Hash-join

Semplice esempio: hash pari/dispari

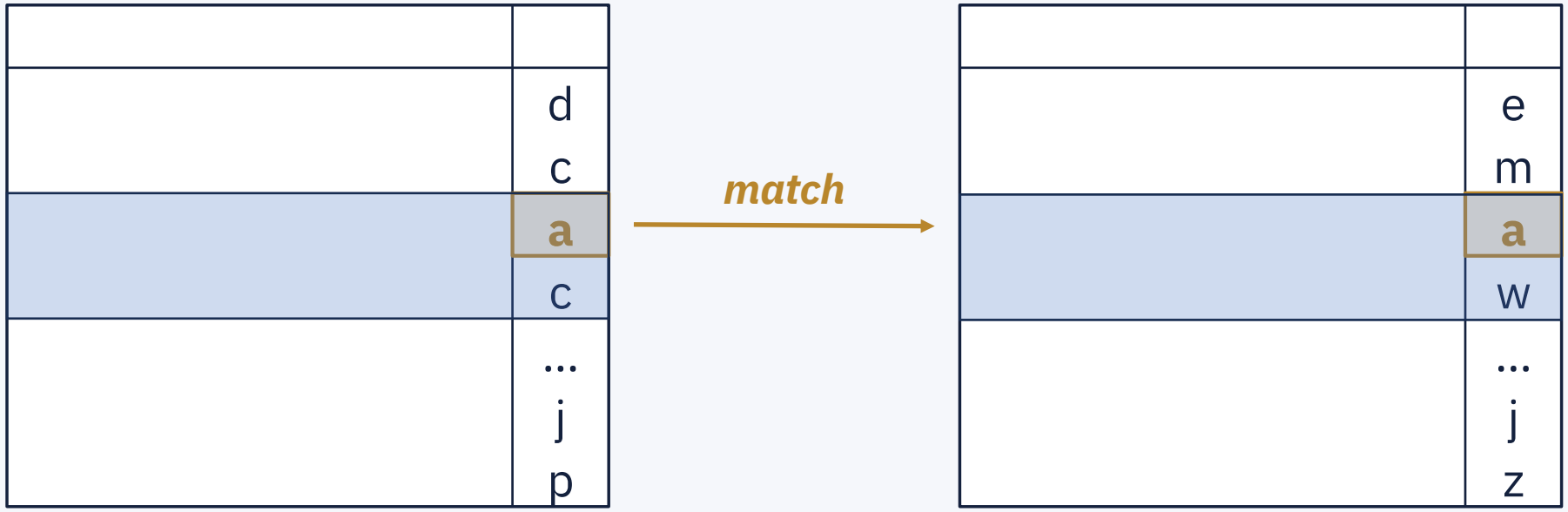


JOIN

3) Hash-join

Bucket Gi — tuple R1,
hash(c) = hash(a) = i

Bucket Hi — tuple R2,
hash(a) = hash(w) = i



Dentro lo stesso bucket si confrontano tutte le coppie (nested-loop locale): solo le chiavi uguali producono una coppia in output.

OTTIMIZZAZIONE DELLE QUERY

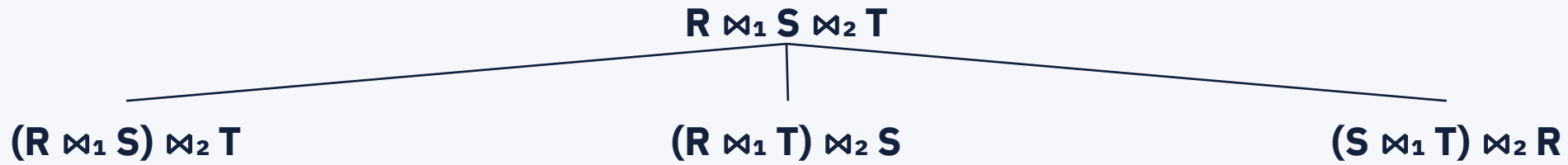
Il processo di ottimizzazione

1. Si costruisce un albero di decisione con le varie alternative ("piani di esecuzione")
2. Si valuta il costo di ciascun piano
3. Si sceglie il piano di costo minore
4. Il cosiddetto "ottimizzatore" trova di solito una "buona" soluzione, non necessariamente l'ottimo

⚠ L'ottimizzatore non dev'essere troppo complesso, o si perde il vantaggio dell'ottimizzazione!

Un albero di decisione

In totale: 75 strategie alternative



Un albero di decisione

